

last time

passing values to/from threads

detach

race conditions and interleaving

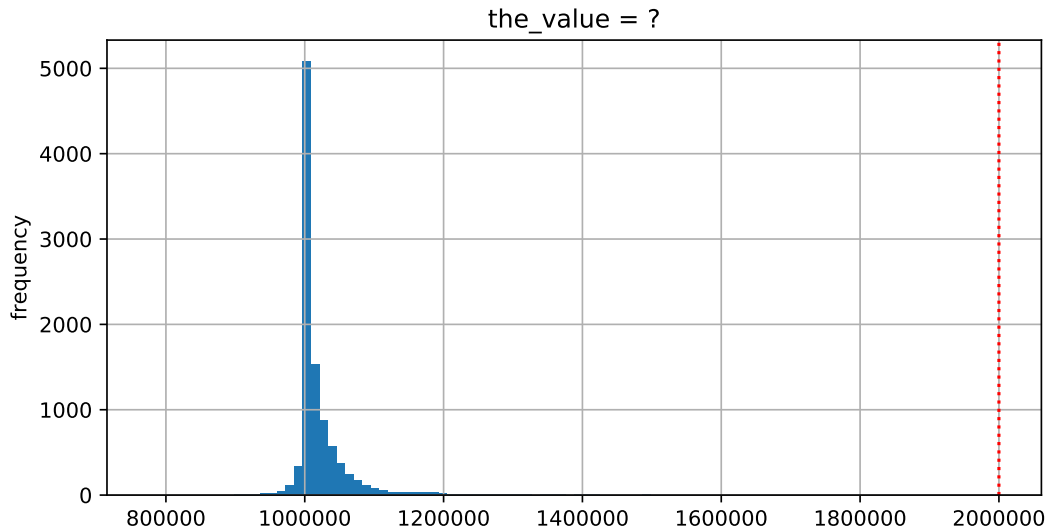
what is/is not atomic

lost adds (program)

```
.global update_loop
update_loop:
    addl $1, the_value // the_value (global variable) += 1
    dec %rdi           // argument 1 -= 1
    jg update_loop     // if argument 1 >= 0 repeat
    ret
```

```
int the_value;
extern void *update_loop(void *);
int main(void) {
    the_value = 0;
    pthread_t A, B;
    pthread_create(&A, NULL, update_loop, (void*) 1000000);
    pthread_create(&B, NULL, update_loop, (void*) 1000000);
    pthread_join(A, NULL); pthread_join(B, NULL);
    // expected result: 1000000 + 1000000 = 2000000
    printf("the_value = %d\n", the_value);
}
```

lost adds (results)



but how?

probably not possible on single core

exceptions can't occur in the middle of add instruction

...but 'add to memory' implemented with multiple steps

still needs to load, add, store internally

can be interleaved with what other cores do

but how?

probably not possible on single core

exceptions can't occur in the middle of add instruction

...but 'add to memory' implemented with multiple steps

still needs to load, add, store internally

can be interleaved with what other cores do

(and actually it's more complicated than that — we'll talk later)

so, what is actually atomic

for now we'll assume: load/stores of 'words'
(64-bit machine = 64-bits words)

in general: processor designer will tell you

their job to design caches, etc. to work as documented

compilers move loads/stores (1)

```
void WaitForReady() {  
    do {} while (!ready);  
}
```

```
WaitForOther:  
    movl ready, %eax    // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2              // while (eax == 0) repeat  
    ...
```


compilers move loads/stores (1)

```
void WaitForReady() {  
    do {} while (!ready);  
}
```

```
WaitForOther:  
    movl ready, %eax    // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2              // while (eax == 0) repeat  
    ...
```

compilers move loads/stores (2)

```
void WaitForOther() {  
    is_waiting = 1;  
    do {} while (!other_ready);  
    is_waiting = 0;  
}
```

WaitForOther:

```
    // compiler optimization: don't set is_waiting to 1,  
    // (why? it will be set to 0 anyway)  
    movl other_ready, %eax // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2 // while (eax == 0) repeat  
    ...  
    movl $0, is_waiting // is_waiting <- 0
```

compilers move loads/stores (2)

```
void WaitForOther() {  
    is_waiting = 1;  
    do {} while (!other_ready);  
    is_waiting = 0;  
}
```

WaitForOther:

```
    // compiler optimization: don't set is_waiting to 1,  
    // (why? it will be set to 0 anyway)  
    movl other_ready, %eax // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2 // while (eax == 0) repeat  
    ...  
    movl $0, is_waiting // is_waiting <- 0
```

compilers move loads/stores (2)

```
void WaitForOther() {  
    is_waiting = 1;  
    do {} while (!other_ready);  
    is_waiting = 0;  
}
```

WaitForOther:

```
// compiler optimization: don't set is_waiting to 1,  
// (why? it will be set to 0 anyway)  
movl other_ready, %eax // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2 // while (eax == 0) repeat  
    ...  
    movl $0, is_waiting // is_waiting <- 0
```

fixing compiler reordering?

isn't there a way to tell compiler not to do these optimizations?

yes, but that is **still not enough!**

processors sometimes do this kind of reordering too (between cores)

pthread and reordering

many pthreads functions **prevent reordering**

everything before function call actually happens before

includes **preventing some optimizations**

e.g. keeping global variable in register for too long

pthread_create, pthread_join, other tools we'll talk about ...

basically: if pthreads is waiting for/starting something, no weird ordering

implementation part 1: prevent compiler reordering

implementation part 2: use special instructions

example: x86 mfence instruction

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time

like updating shared balance

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time

like updating shared balance

critical section: code that exactly one thread can execute at a time

result of mutual exclusion

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time

like updating shared balance

critical section: code that exactly one thread can execute at a time

result of mutual exclusion

lock: object only one thread can hold at a time

interface for creating critical sections

lock analogy

agreement: only change account balances while wearing this hat

normally hat kept on table

put on hat when editing balance

hopefully, only one person (= thread) can wear hat a time

need to wait for them to remove hat to put it on

lock analogy

agreement: only change account balances while wearing this hat

normally hat kept on table

put on hat when editing balance

hopefully, only one person (= thread) can wear hat a time

need to wait for them to remove hat to put it on

“lock (or acquire) the lock” = get and put on hat

“unlock (or release) the lock” = put hat back on table

the lock primitive

locks: an object with (at least) two operations:

acquire or *lock* — wait until lock is free, then “grab” it

release or *unlock* — let others use lock, wakeup waiters

typical usage: everyone acquires lock before using shared resource

forget to acquire lock? weird things happen

```
Lock(account_lock);  
balance += ...;  
Unlock(account_lock);
```

the lock primitive

locks: an object with (at least) two operations:

acquire or *lock* — **wait** until lock is free, then “grab” it

release or *unlock* — let others use lock, wakeup waiters

typical usage: everyone acquires lock before using shared resource

forget to acquire lock? weird things happen

```
Lock(account_lock);  
balance += ...;  
Unlock(account_lock);
```

waiting for lock?

when waiting — ideally:

not using processor (at least if waiting a while)

OS can context switch to other programs

pthread mutex

```
#include <pthread.h>
```

```
pthread_mutex_t account_lock;  
pthread_mutex_init(&account_lock, NULL);  
    // or: pthread_mutex_t account_lock =  
    //      PTHREAD_MUTEX_INITIALIZER;  
...  
pthread_mutex_lock(&account_lock);  
balance += ...;  
pthread_mutex_unlock(&account_lock);
```

exercise

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init_one", two = "init_two";
void ThreadA() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadB"; // (B1)
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

possible values of one/two after A+B run?

exercise (alternate 1)

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init_one", two = "init_two";
void ThreadA() {
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadB"; // (B1)
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

possible values of one/two after A+B run?

exercise (alternate 2)

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init_one", two = "init_two";
void ThreadA() {
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadB"; // (B1)
    pthread_mutex_unlock(&lock1);
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
}
```

possible values of one/two after A+B run?

POSIX mutex restrictions

pthread_mutex rule: unlock from same thread you lock in

does this actually matter?

depends on how pthread_mutex is implemented

preview: general sync

lots of coordinating threads beyond locks

will talk about two general tools later:k

- monitors/condition variables

- semaphores [if time]

big added feature: wait for arbitrary thing to happen

also some less general tools: barriers

a bad idea

one **bad** idea to wait for an event:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; bool ready = false;
void WaitForReady() {
    pthread_mutex_lock(&lock);
    do {
        pthread_mutex_unlock(&lock);
        /* only time MarkReady() can run */
        pthread_mutex_lock(&lock);
    } while (!ready);
    pthread_mutex_unlock(&lock);
}
void MarkReady() {
    pthread_mutex_lock(&lock);
    ready = true;
    pthread_mutex_unlock(&lock);
}
```

wastes processor time; MarkReady can stall waiting for unlock window

beyond locks

in practice: want more than locks for synchronization

for waiting for arbitrary events (without CPU-hogging-loop):

- monitors

- semaphores

for common synchronization patterns:

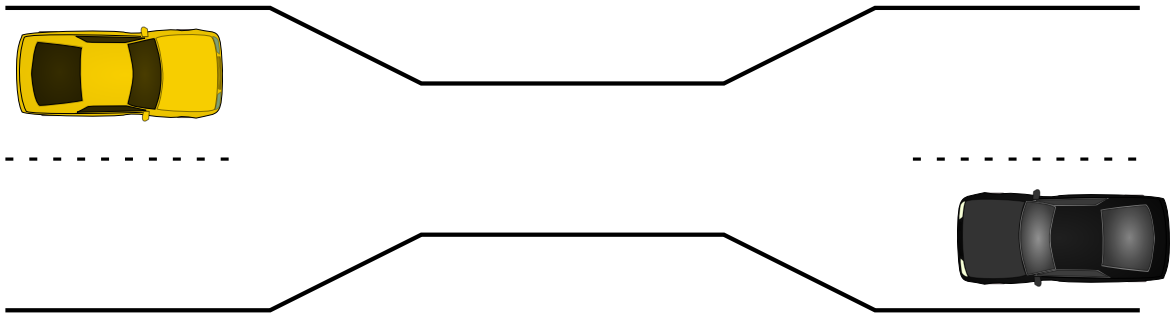
- barriers

- reader-writer locks

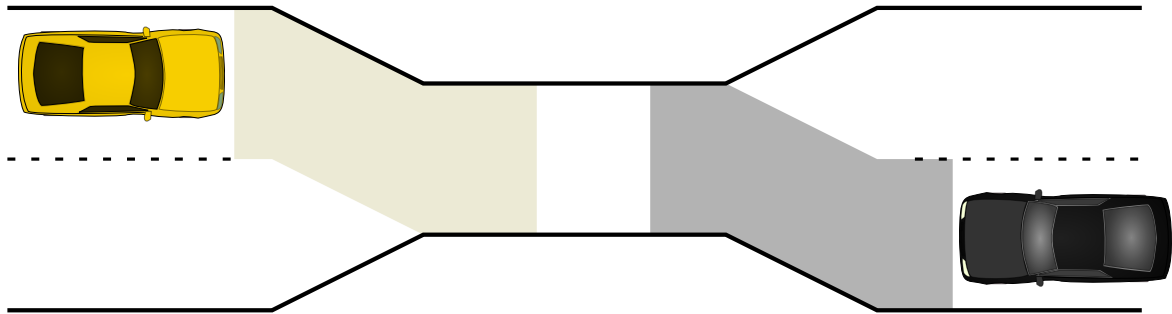
higher-level interface:

- transactions

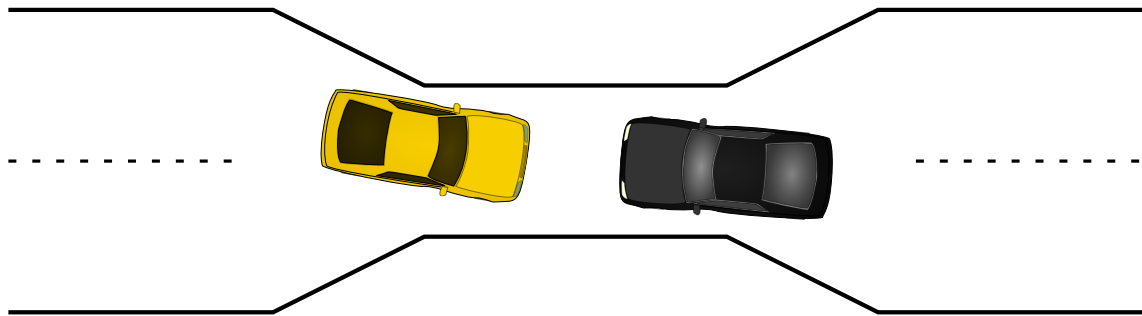
the one-way bridge



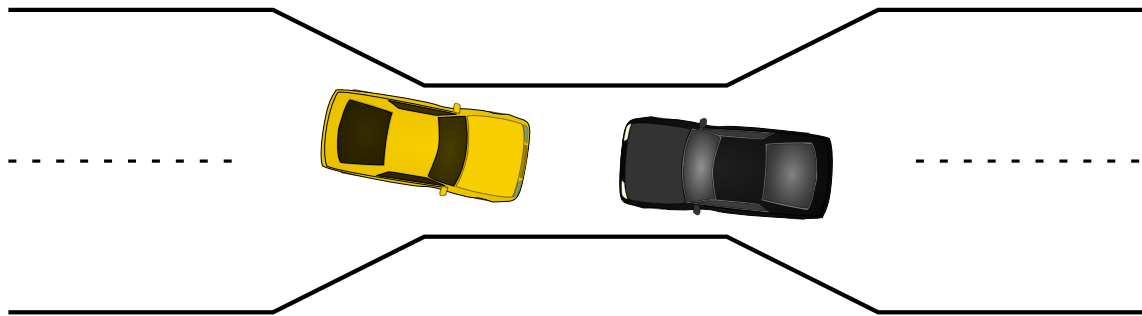
the one-way bridge



the one-way bridge



the one-way bridge



moving two files

```
struct Dir {  
    mutex_t lock; HashMap entries;  
};  
void MoveFile(Dir *from_dir, Dir *to_dir, string filename) {  
    mutex_lock(&from_dir->lock);  
    mutex_lock(&to_dir->lock);  
  
    Map_put(to_dir->entries, filename,  
            Map_get(from_dir->entries, filename));  
    Map_erase(from_dir->entries, filename);  
  
    mutex_unlock(&to_dir->lock);  
    mutex_unlock(&from_dir->lock);  
}
```

Thread 1: MoveFile(A, B, "foo")

Thread 2: MoveFile(B, A, "bar")

moving two files: lucky timeline (1)

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock);

(do move)

unlock(&B->lock);

unlock(&A->lock);

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock);

lock(&A->lock);

(do move)

unlock(&B->lock);

unlock(&A->lock);

moving two files: lucky timeline (2)

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock);

(do move)

unlock(&B->lock);

unlock(&A->lock);

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock...

(waiting for B lock)

lock(&B->lock);

lock(&A->lock...

lock(&A->lock);

(do move)

unlock(&A->lock);

unlock(&B->lock);

moving two files: unlucky timeline

Thread 1

```
MoveFile(A, B, "foo")
```

```
lock(&A->lock);
```

Thread 2

```
MoveFile(B, A, "bar")
```

```
lock(&B->lock);
```

moving two files: unlucky timeline

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock... stalled

(waiting for lock on B)

(waiting for lock on B)

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock);

lock(&A->lock... stalled

(waiting for lock on A)

moving two files: unlucky timeline

Thread 1

```
MoveFile(A, B, "foo")
```

```
lock(&A->lock);
```

```
lock(&B->lock... stalled
```

```
(waiting for lock on B)
```

```
(waiting for lock on B)
```

```
(do move) unreachable
```

```
unlock(&B->lock); unreachable
```

```
unlock(&A->lock); unreachable
```

Thread 2

```
MoveFile(B, A, "bar")
```

```
lock(&B->lock);
```

```
lock(&A->lock... stalled
```

```
(waiting for lock on A)
```

```
(do move) unreachable
```

```
unlock(&A->lock); unreachable
```

```
unlock(&B->lock); unreachable
```


moving two files: unlucky timeline

Thread 1

```
MoveFile(A, B, "foo")
```

```
lock(&A->lock);
```

```
lock(&B->lock... stalled
```

```
(waiting for lock on B)
```

```
(waiting for lock on B)
```

```
(do move) unreachable
```

```
unlock(&B->lock); unreachable
```

```
unlock(&A->lock); unreachable
```

Thread 2

```
MoveFile(B, A, "bar")
```

```
lock(&B->lock);
```

```
lock(&A->lock... stalled
```

```
(waiting for lock on A)
```

```
(do move) unreachable
```

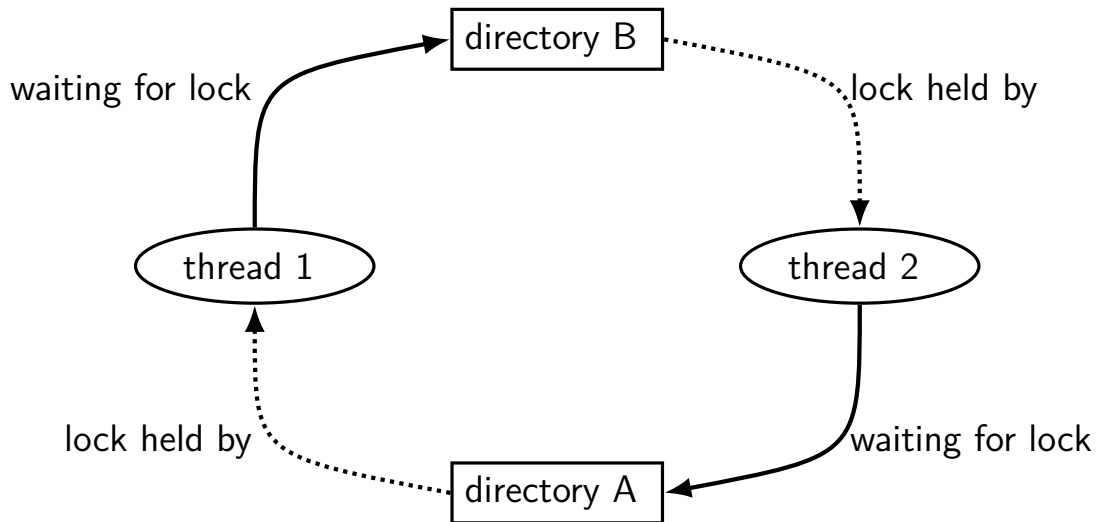
```
unlock(&A->lock); unreachable
```

```
unlock(&B->lock); unreachable
```

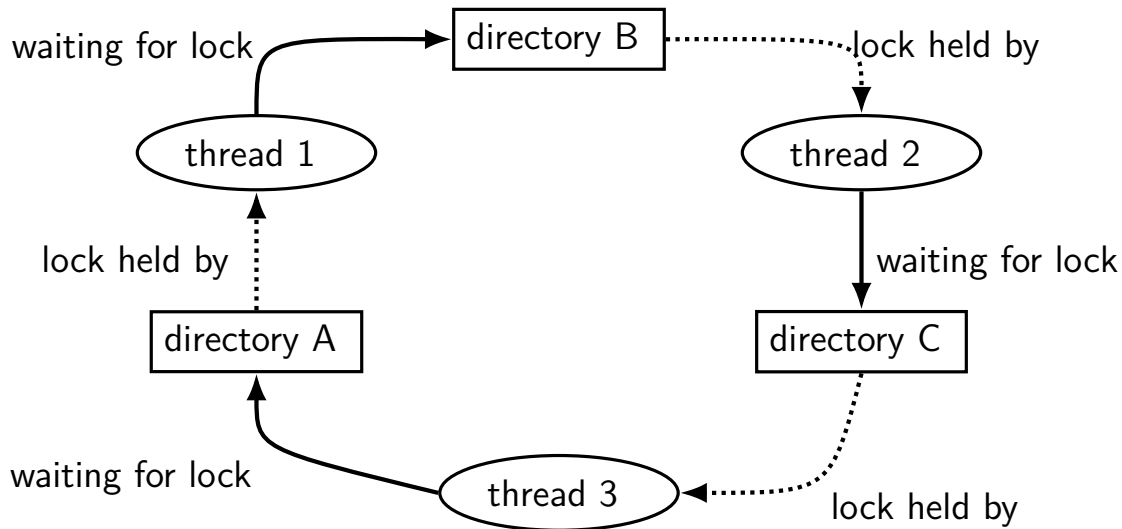
Thread 1 holds A lock, waiting for Thread 2 to release B lock

Thread 2 holds B lock, waiting for Thread 1 to release A lock

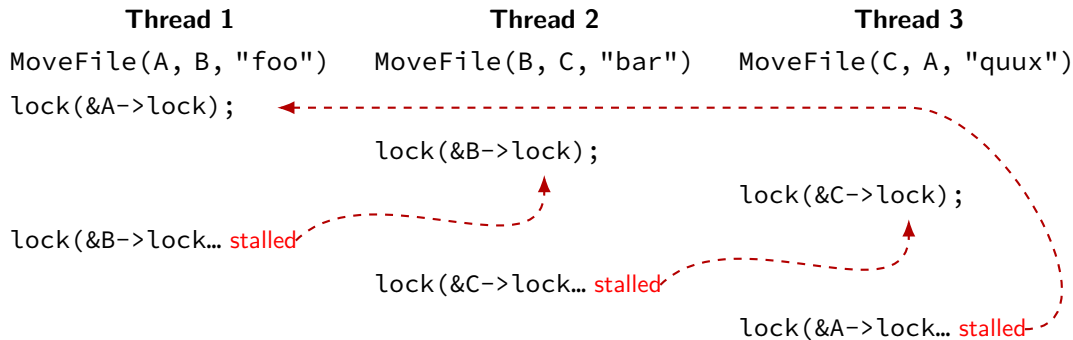
moving two files: dependencies



moving three files: dependencies



moving three files: unlucky timeline



deadlock with free space

Thread 1

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB)
Free(1 MB)
```

Thread 2

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB)
Free(1 MB)
```

2 MB of space — deadlock possible with unlucky order

deadlock with free space (unlucky case)

Thread 1

AllocateOrWaitFor(1 MB)

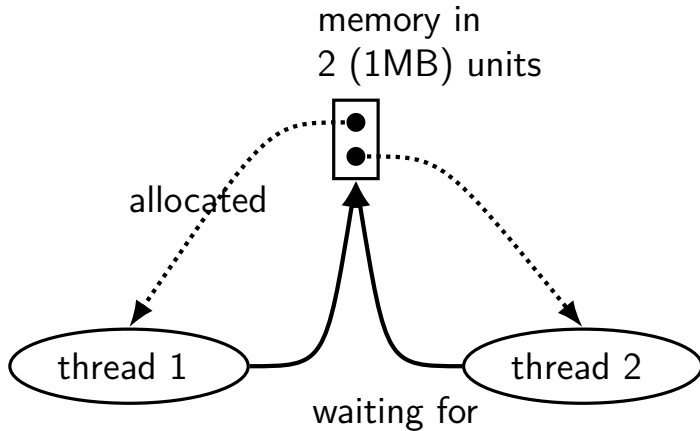
AllocateOrWaitFor(1 MB... stalled

Thread 2

AllocateOrWaitFor(1 MB)

AllocateOrWaitFor(1 MB... stalled

free space: dependency graph



deadlock with free space (lucky case)

Thread 1

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB);
Free(1 MB);
```

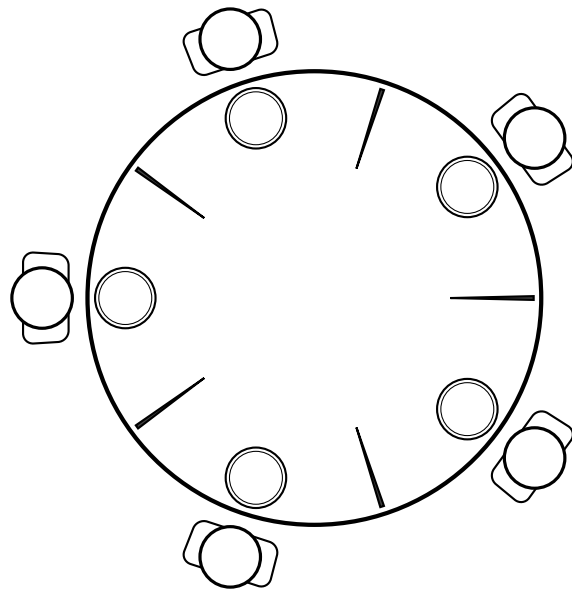
Thread 2

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB);
Free(1 MB);
```


lab next week

applying solutions to deadlock to classic *dining philosophers* problem

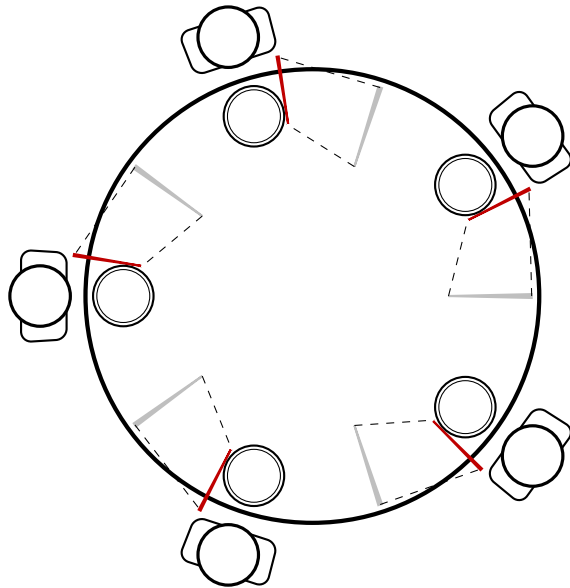
dining philosophers



five philosophers either think or eat
to eat:

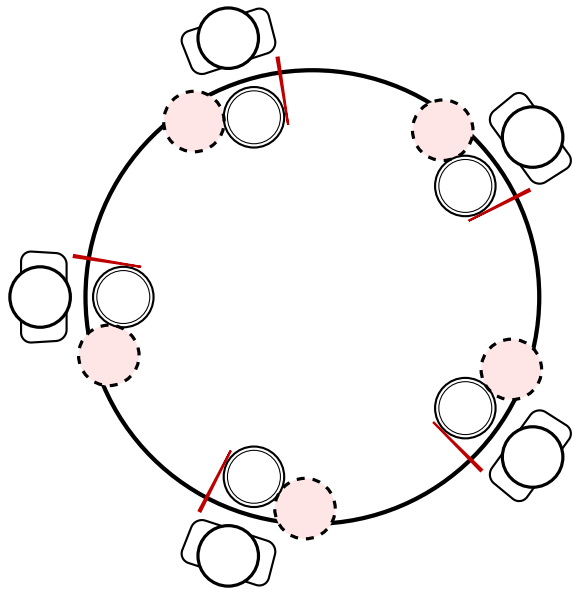
grab chopstick on left, then
grab chopstick on right, then
then eat, then
return chopsticks

dining philosophers



everyone eats at the same time?
grab left chopstick, then...

dining philosophers



everyone eats at the same time?
grab left chopstick, then
try to grab right chopstick, ...
we're at an impasse

deadlock

deadlock — circular waiting for resources

resource = something needed by a thread to do work

- locks

- CPU time

- disk space

- memory

- ...

often non-deterministic in practice

most common example: **when acquiring multiple locks**

deadlock

deadlock — circular waiting for **resources**

resource = something needed by a thread to do work

- locks

- CPU time

- disk space

- memory

- ...

often non-deterministic in practice

most common example: **when acquiring multiple locks**

deadlock requirements

mutual exclusion

one thread at a time can use a resource

hold and wait

thread holding a resources waits to acquire *another* resource

no preemption of resources

resources are only released voluntarily

thread trying to acquire resources can't 'steal'

circular wait

there exists a set $\{T_1, \dots, T_n\}$ of waiting threads such that

T_1 is waiting for a resource held by T_2

T_2 is waiting for a resource held by T_3

...

T_n is waiting for a resource held by T_1

how is deadlock possible?

Given list: A, B, C, D, E

```
RemoveNode(LinkedListNode *node) {  
    pthread_mutex_lock(&node->lock);  
    pthread_mutex_lock(&node->prev->lock);  
    pthread_mutex_lock(&node->next->lock);  
    node->next->prev = node->prev; node->prev->next = node->next;  
    pthread_mutex_unlock(&node->next->lock); pthread_mutex_unlock(&node->prev->lock);  
    pthread_mutex_unlock(&node->lock);  
}
```

Which of these (all run in parallel) can deadlock?

- A. RemoveNode(B) and RemoveNode(C)
- B. RemoveNode(B) and RemoveNode(D)
- C. RemoveNode(B) and RemoveNode(C) and RemoveNode(D)
- D. A and C
- E. B and C
- F. all of the above
- G. none of the above

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request **all resources at once**

no *hold and wait*

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request **all resources at once**

no *hold and wait*

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request **all resources at once**

no *hold and wait*

deadlock prevention techniques

infinite resources

memory allocation: malloc() fails rather than waiting (no deadlock)

locks: pthread_mutex_trylock fails rather than waiting

problem: retry how many times? **no bound on number of tries needed**

...

exclusion

exclusion

no waiting

“busy signal” — abort and (maybe) retry

revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“**busy signal**” — **abort and (maybe) retry**
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request **all resources at once**

no *hold and wait*

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no share

requires some way to undo partial changes to avoid errors
common approach for databases

exclusion

no waiti

...

“busy signal” — abort and (maybe) retry

*no hold and wait/
preemption*

revoke/preempt resources

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request **all resources at once**

no *hold and wait*

acquiring locks in consistent order (1)

```
MoveFile(Dir* from_dir, Dir* to_dir, string filename) {  
    if (from_dir->path < to_dir->path) {  
        lock(&from_dir->lock);  
        lock(&to_dir->lock);  
    } else {  
        lock(&to_dir->lock);  
        lock(&from_dir->lock);  
    }  
    ...  
}
```


acquiring locks in consistent order (1)

```
MoveFile(Dir* from_dir, Dir* to_dir, string filename) {  
    if (from_dir->path < to_dir->path) {  
        lock(&from_dir->lock);  
        lock(&to_dir->lock);  
    } else {  
        lock(&to_dir->lock);  
        lock(&from_dir->lock);  
    }  
    ...  
}
```

any ordering will do
e.g. compare pointers

acquiring locks in consistent order (2)

often by convention, e.g. Linux kernel comments:

```
/*  
 * ...  
 * Lock order:  
 *     contex.ldt_usr_sem  
 *     mmap_sem  
 *     context.lock  
 */
```

```
/*  
 * ...  
 * Lock order:  
 * 1. slab_mutex (Global Mutex)  
 * 2. node->list_lock  
 * 3. slab_lock(page) (Only on some arches and for debugging)  
 * ...  
 */
```

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request **all resources at once**

no *hold and wait*

barriers

compute minimum of 100M element array with 2 processors

algorithm:

compute minimum of 50M of the elements on each CPU

one thread for each CPU

wait for all computations to finish

take minimum of all the minimums

barriers

compute minimum of 100M element array with 2 processors

algorithm:

compute minimum of 50M of the elements on each CPU
one thread for each CPU

wait for all computations to finish

take minimum of all the minimums

barriers API

`barrier.Initialize(NumberOfThreads)`

`barrier.Wait()` — return after all threads have waited

idea: multiple threads perform computations in parallel

threads wait for **all other threads** to call `Wait()`

barrier: waiting for finish

```
barrier.Initialize(2);
```

Thread 0

```
partial_mins[0] =  
    /* min of first  
       50M elems */;
```

```
barrier.Wait();
```

```
total_min = min(  
    partial_mins[0],  
    partial_mins[1]  
);
```

Thread 1

```
partial_mins[1] =  
    /* min of last  
       50M elems */  
barrier.Wait();
```

barriers: reuse

Thread 0

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

Thread 1

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```


barriers: reuse

Thread 0

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

Thread 1

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

barriers: reuse

Thread 0

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

Thread 1

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

pthread barriers

```
pthread_barrier_t barrier;  
pthread_barrier_init(  
    &barrier,  
    NULL /* attributes */,  
    numberOfThreads  
);  
...  
...  
pthread_barrier_wait(&barrier);
```

exercise

```
pthread_barrier_t barrier; int x = 0, y = 0;
void thread_one() {
    y = 10;
    pthread_barrier_wait(&barrier);
    y = x + y;
    pthread_barrier_wait(&barrier);
    pthread_barrier_wait(&barrier);
    printf("%d_%d\n", x, y);
}
void thread_two() {
    x = 20;
    pthread_barrier_wait(&barrier);
    pthread_barrier_wait(&barrier);
    x = x + y;
    pthread_barrier_wait(&barrier);
}
```

output? (if both run at once, barrier set for 2 threads)

life homework (pseudocode)

```
for (int time = 0; time < MAX_ITERATIONS; ++time) {  
    for (int y = 0; y < size; ++y) {  
        for (int x = 0; x < size; ++x) {  
            to_grid(x, y) = computeValue(from_grid, x, y);  
        }  
    }  
    swap(from_grid, to_grid);  
}
```

life homework

compute grid of values for time t from grid for time $t - 1$

compute new value at i, j based on surrounding values

parallel version: produce parts of grid in different threads

use barriers to finish time t before going to time $t + 1$

monitors/condition variables

locks for mutual exclusion

condition variables for waiting for event

represents **list of waiting threads**

operations: wait (for event); signal/broadcast (that event happened)

related data structures

monitor = lock + 0 or more condition variables + shared data

Java: every object is a monitor (has instance variables, built-in lock, cond. var)

python: build your own: provides you locks + condition variables

monitor idea

a monitor

lock
shared data
condvar 1
condvar 2
...
operation1(...)
operation2(...)

monitor idea

a monitor

lock
shared data
condvar 1
condvar 2
...
operation1(...)
operation2(...)

lock must be acquired
before accessing
any part of monitor's stuff

monitor idea

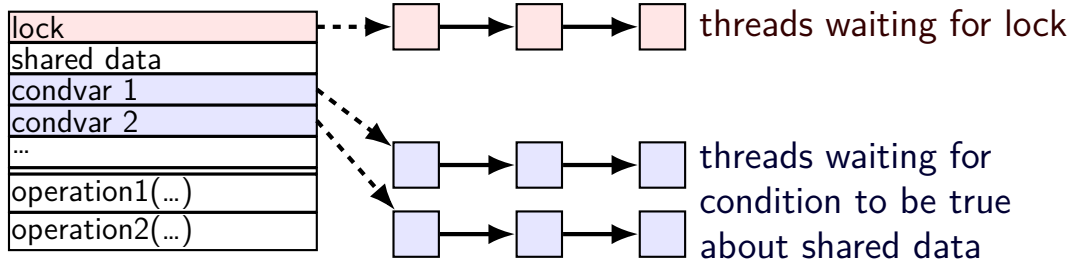
a monitor

lock
shared data
condvar 1
condvar 2
...
operation1(...)
operation2(...)



monitor idea

a monitor



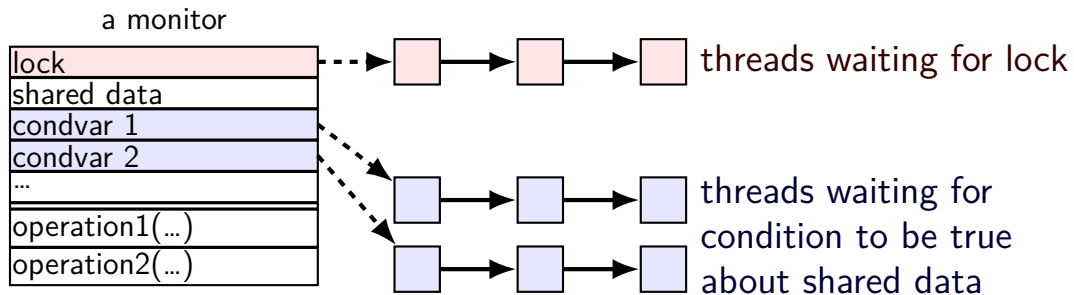
condvar operations

condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue
...and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



condvar operations

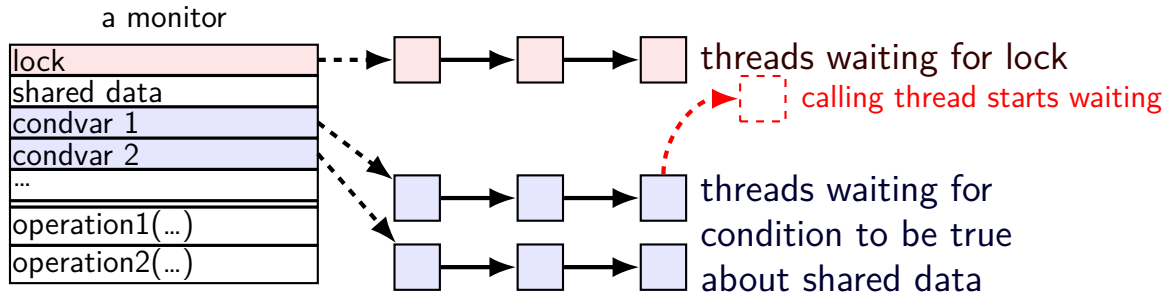
condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue

...and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



condvar operations

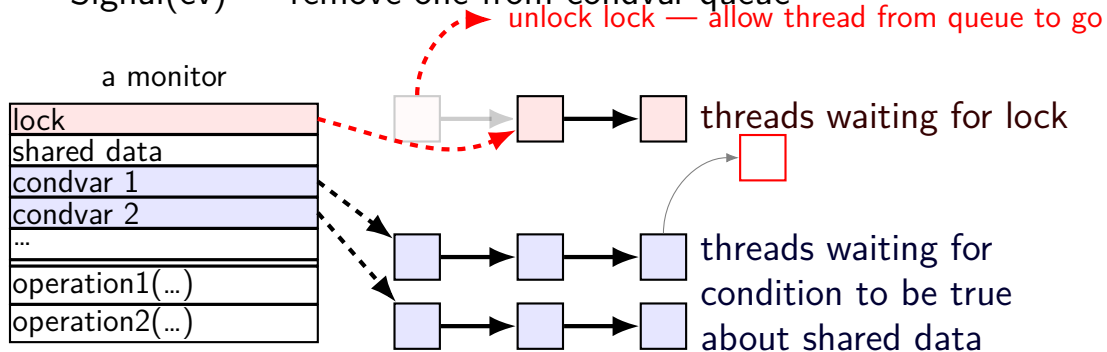
condvar operations:

Wait(cv, lock) — **unlock** lock, add current thread to cv queue

...and **reacquire** lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



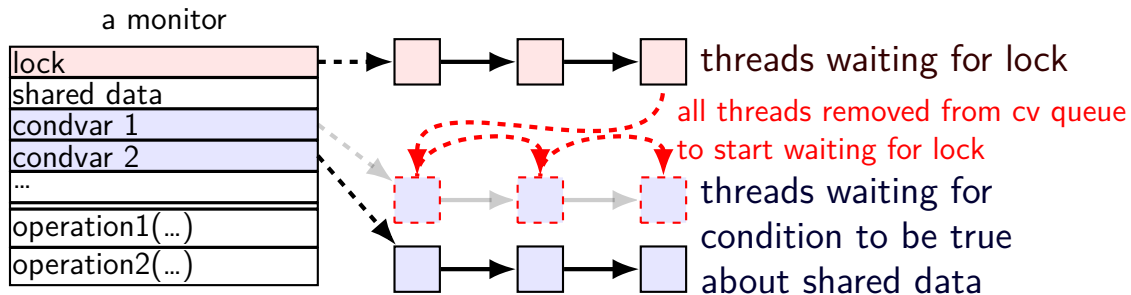
condvar operations

condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue
...and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



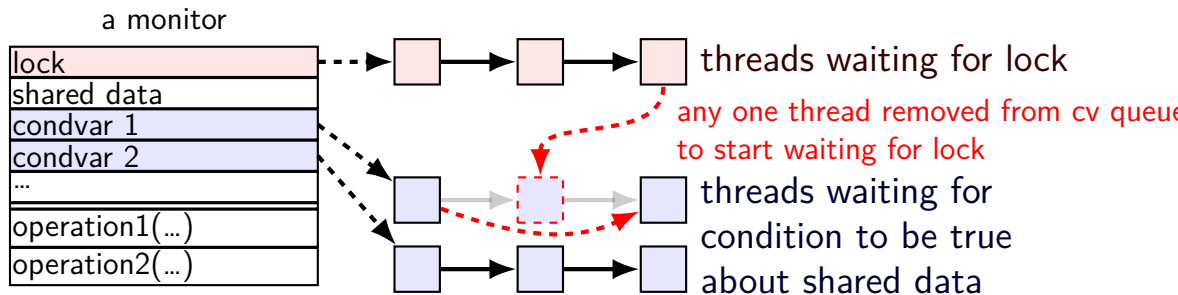
condvar operations

condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue
...and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



pthread cv usage

// MISSING: init calls, etc.

```
pthread_mutex_t lock;  
bool finished;    // data, only accessed with after acquiring lock  
pthread_cond_t finished_cv; // to wait for 'finished' to be true
```

```
void WaitForFinished() {  
    pthread_mutex_lock(&lock);  
    while (!finished) {  
        pthread_cond_wait(&finished_cv, &lock);  
    }  
    pthread_mutex_unlock(&lock);  
}
```

```
void Finish() {  
    pthread_mutex_lock(&lock);  
    finished = true;  
    pthread_cond_broadcast(&finished_cv);  
    pthread_mutex_unlock(&lock);  
}
```

pthread cv usage

// MISSING: init calls, etc.

```
pthread_mutex_t lock;
```

```
bool finished;    // data, only accessed with after acquiring lock
```

```
pthread_cond_t finished_cv; // to wait for 'finished' to be true
```

```
void WaitForFinished() {
```

```
    pthread_mutex_lock(&lock);
```

```
    while (!finished) {
```

```
        pthread_cond_wait(&finished_cv, &lock);
```

```
    }
```

```
    pthread_mutex_unlock(&lock);
```

```
}
```

acquire lock before
reading or writing finished

```
void Finish() {
```

```
    pthread_mutex_lock(&lock);
```

```
    finished = true;
```

```
    pthread_cond_broadcast(&finished_cv);
```

```
    pthread_mutex_unlock(&lock);
```

```
}
```

pthread cv usage

// MISSING: init calls, etc.

```
pthread_mutex_t lock;
```

```
bool finished;    // data, only accessed with after acquiring lock
```

```
pthread_cond_t finished_cv; // to wait for 'finished' to be true
```

```
void WaitForFinished() {  
    pthread_mutex_lock(&lock);  
    while (!finished) {  
        pthread_cond_wait(&finished_cv, &lock);  
    }  
    pthread_mutex_unlock(&lock);  
}
```

check whether we need to wait at all
(why a loop? we'll explain later)

```
void Finish() {  
    pthread_mutex_lock(&lock);  
    finished = true;  
    pthread_cond_broadcast(&finished_cv);  
    pthread_mutex_unlock(&lock);  
}
```

pthread cv usage

// MISSING: init calls, etc.

```
pthread_mutex_t lock;
```

```
bool finished;    // data, only accessed with after acquiring lock
```

```
pthread_cond_t finished_cv; // to wait for 'finished' to be true
```

```
void WaitForFinished() {
```

```
    pthread_mutex_lock(&lock);
```

```
    while (!finished) {
```

```
        pthread_cond_wait(&finished_cv, &lock);
```

```
    }
```

```
    pthread_mutex_unlock(&lock);
```

```
}
```

```
void Finish() {
```

```
    pthread_mutex_lock(&lock);
```

```
    finished = true;
```

```
    pthread_cond_broadcast(&finished_cv);
```

```
    pthread_mutex_unlock(&lock);
```

```
}
```

know we need to wait
(finished can't change while we have lock)
so wait, releasing lock...

pthread cv usage

// MISSING: init calls, etc.

```
pthread_mutex_t lock;
```

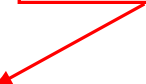
```
bool finished;    // data, only accessed with after acquiring lock
```

```
pthread_cond_t finished_cv; // to wait for 'finished' to be true
```

```
void WaitForFinished() {  
    pthread_mutex_lock(&lock);  
    while (!finished) {  
        pthread_cond_wait(&finished_cv, &lock);  
    }  
    pthread_mutex_unlock(&lock);  
}
```

```
void Finish() {  
    pthread_mutex_lock(&lock);  
    finished = true;  
    pthread_cond_broadcast(&finished_cv);  
    pthread_mutex_unlock(&lock);  
}
```

allow all waiters to proceed
(once we unlock the lock)



WaitForFinish timeline 1

WaitForFinish thread	Finish thread
<code>mutex_lock(&lock)</code> (thread has lock)	
	<code>mutex_lock(&lock)</code> (start waiting for lock)
<code>while (!finished) ...</code> <code>cond_wait(&finished_cv, &lock);</code> (start waiting for cv)	(done waiting for lock)
	<code>finished = true</code> <code>cond_broadcast(&finished_cv)</code>
(done waiting for cv) (start waiting for lock)	
	<code>mutex_unlock(&lock)</code>
(done waiting for lock) <code>while (!finished) ...</code> (finished now true, so return) <code>mutex_unlock(&lock)</code>	

WaitForFinish timeline 2

WaitForFinish thread	Finish thread
	<code>mutex_lock(&lock)</code> <code>finished = true</code> <code>cond_broadcast(&finished_cv)</code> <code>mutex_unlock(&lock)</code>
<code>mutex_lock(&lock)</code> <code>while (!finished) ...</code> (finished now true, so return) <code>mutex_unlock(&lock)</code>	

why the loop

```
while (!finished) {  
    pthread_cond_wait(&finished_cv, &lock);  
}
```

we only broadcast if finished is true

so why check finished afterwards?

why the loop

```
while (!finished) {  
    pthread_cond_wait(&finished_cv, &lock);  
}
```

we only broadcast if finished is true

so why check finished afterwards?

pthread_cond_wait manual page:

“**Spurious wakeups** ... may occur.”

spurious wakeup = wait returns even though nothing happened

unbounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}  
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

unbounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

rule: never touch buffer
without acquiring lock

otherwise: what if two threads
simultaneously en/dequeue?
(both use same array/linked list entry?)
(both reallocate array?)

unbounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

check if empty
if so, dequeue

okay because have lock

other threads cannot dequeue here

unbounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

wake one Consume thread
if any are waiting

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

unbounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

Thread 1

Produce()
...lock
...enqueue
...signal
...unlock

Thread 2

Consume()
...lock
...empty? no
...dequeue
...unlock
return

0 iterations: Produce() called before Consume()
1 iteration: Produce() signalled, probably
2+ iterations: spurious wakeup or ...?

unbounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

Thread 1

Thread 2

	Consume()
	...lock
	...empty? yes
	...unlock/start wait
Produce()	waiting for data_ready
...lock	
...enqueue	
...signal	stop wait
...unlock	lock
	...empty? no
	...dequeue
	...unlock
	return

0 iterations: Produce() called before Consume()
1 iteration: Produce() signalled, probably
2+ iterations: spurious wakeup or ...?

unbounded buffer producer/consumer

```
pthread_mutex_t lock;
pthread_cond_t data_ready;
UnboundedQueue buffer;
```

```
Produce(item) {
    pthread_mutex_lock(&lock);
    buffer.enqueue(item);
    pthread_cond_signal(&data_ready);
    pthread_mutex_unlock(&lock);
}
```

```
Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.empty()) {
        pthread_cond_wait(&data_ready, &lock);
    }
    item = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return item;
}
```

Thread 1

Produce()
...lock
...enqueue
...signal
...unlock

Thread 2

Consume()
...lock
...empty? yes
...unlock/start wait
waiting for data_ready
stop wait
waiting for lock
...lock
...empty? yes
...unlock/start wait

Thread 3

Consume()
waiting for lock
lock
...empty? no
...dequeue
...unlock
return

0 iterations: Produce() called before Consume()
 1 iteration: Produce() signalled, probably
 2+ iterations: spurious wakeup or ...?

unbounded buffer producer/consumer

```
pthread_mutex_t lock;
pthread_cond_t data_ready;
UnboundedQueue buffer;
```

in pthreads: signalled thread not
guaranteed to hold lock next

alternate design:
signalled thread gets lock next
called "Hoare scheduling"
not done by pthreads, Java, ...

```
pthread_cond_wait(&data_ready, &lock);
}
item = buffer.dequeue();
pthread_mutex_unlock(&lock);
return item;
}
```

Thread 1

```
Produce()
...lock
...enqueue
...signal
...unlock
```

Thread 2

```
Consume()
...lock
...empty? yes
...unlock/start wait

waiting for
data_ready

stop wait

waiting for
lock

...lock
...empty? yes
...unlock/start wait
```

Thread 3

```
Consume()
waiting for
lock

lock
...empty? no
...dequeue
...unlock
return
```

0 iterations: Produce() called before Consume()
1 iteration: Produce() signalled, probably
2+ iterations: spurious wakeup or ...?

Hoare versus Mesa monitors

Hoare-style monitors

- signal 'hands off' lock to awoken thread

Mesa-style monitors

- any eligible thread gets lock next
(maybe some other idea of priority?)

every current threading library I know of does Mesa-style

bounded buffer producer/consumer

```
pthread_mutex_t lock;
pthread_cond_t data_ready; pthread_cond_t space_ready;
BoundedQueue buffer;
Produce(item) {
    pthread_mutex_lock(&lock);
    while (buffer.full()) { pthread_cond_wait(&space_ready, &lock); }
    buffer.enqueue(item);
    pthread_cond_signal(&data_ready);
    pthread_mutex_unlock(&lock);
}
Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.empty()) {
        pthread_cond_wait(&data_ready, &lock);
    }
    item = buffer.dequeue();
    pthread_cond_signal(&space_ready);
    pthread_mutex_unlock(&lock);
    return item;
}
```

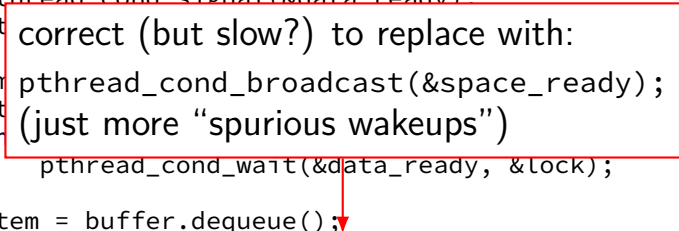
bounded buffer producer/consumer

```
pthread_mutex_t lock;
pthread_cond_t data_ready; pthread_cond_t space_ready;
BoundedQueue buffer;
Produce(item) {
    pthread_mutex_lock(&lock);
    while (buffer.full()) { pthread_cond_wait(&space_ready, &lock); }
    buffer.enqueue(item);
    pthread_cond_signal(&data_ready);
    pthread_mutex_unlock(&lock);
}
Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.empty()) {
        pthread_cond_wait(&data_ready, &lock);
    }
    item = buffer.dequeue();
    pthread_cond_signal(&space_ready);
    pthread_mutex_unlock(&lock);
    return item;
}
```

bounded buffer producer/consumer

```
pthread_mutex_t lock;
pthread_cond_t data_ready; pthread_cond_t space_ready;
BoundedQueue buffer;
Produce(item) {
    pthread_mutex_lock(&lock);
    while (buffer.full()) { pthread_cond_wait(&space_ready, &lock); }
    buffer.enqueue(item);
    pthread_cond_signal(&data_ready);
}
Consumption {
    pthread_cond_wait(&data_ready, &lock);
    item = buffer.dequeue();
    pthread_cond_signal(&space_ready);
    pthread_mutex_unlock(&lock);
    return item;
}
```

correct (but slow?) to replace with:
pthread_cond_broadcast(&space_ready);
(just more “spurious wakeups”)



bounded buffer producer/consumer

```
pthread_mutex_t lock;
pthread_cond_t data_ready; pthread_cond_t space_ready;
BoundedQueue buffer;
Produce(item) {
    pthread_mutex_lock(&lock);
    while (buffer.full()) { pthread_cond_wait(&space_ready, &lock); }
    buffer.enqueue(item);
    pthread_cond_signal(&data_ready);
    pthread_mutex_unlock(&lock);
}
Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.empty()) {
        pthread_cond_wait(&data_ready, &lock);
    }
    item = buffer.dequeue();
    pthread_cond_signal(&space_ready);
    pthread_mutex_unlock(&lock);
    return item;
}
```

correct but slow to replace
data_ready and space_ready
with 'combined' condvar ready
and use broadcast
(just more "spurious wakeups")

monitor pattern

```
pthread_mutex_lock(&lock);
while (!condition A) {
    pthread_cond_wait(&condvar_for_A, &lock);
}
... /* manipulate shared data, changing other conditions */
if (set condition A) {
    pthread_cond_broadcast(&condvar_for_A);
    /* or signal, if only one thread cares */
}
if (set condition B) {
    pthread_cond_broadcast(&condvar_for_B);
    /* or signal, if only one thread cares */
}
...
pthread_mutex_unlock(&lock)
```

monitors rules of thumb

never touch shared data without holding the lock

keep lock held for **entire operation**:

verifying condition (e.g. buffer not full) *up to and including*
manipulating data (e.g. adding to buffer)

create condvar for every kind of scenario waited for

always write **loop** calling cond_wait to wait for condition X

broadcast/signal condition variable **every time you change X**

monitors rules of thumb

never touch shared data without holding the lock

keep lock held for **entire operation**:

verifying condition (e.g. buffer not full) *up to and including*
manipulating data (e.g. adding to buffer)

create condvar for every kind of scenario waited for

always write **loop** calling cond_wait to wait for condition X

broadcast/signal condition variable **every time you change X**

correct but slow to...

broadcast when just signal would work

broadcast or signal when nothing changed

use one condvar for multiple conditions

mutex/cond var init/destroy

```
pthread_mutex_t mutex;  
pthread_cond_t cv;  
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cv, NULL);  
// --OR--  
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;  
  
// and when done:  
...  
pthread_cond_destroy(&cv);  
pthread_mutex_destroy(&mutex);
```

wait for both finished

```
// MISSING: init calls, etc.
```

```
pthread_mutex_t lock;  
bool finished[2];  
pthread_cond_t both_finished_cv;
```

```
void WaitForBothFinished() {  
    pthread_mutex_lock(&lock);  
    while (_____) {  
        pthread_cond_wait(&both_finished_cv, &lock);  
    }  
    pthread_mutex_unlock(&lock);  
}
```

```
void Finish(int index) {  
    pthread_mutex_lock(&lock);  
    finished[index] = true;  
    _____  
    pthread_mutex_unlock(&lock);  
}
```

wait for both finished

// MISSING: init calls, etc.

```
pthread_mutex_t lock;  
bool finished[2];  
pthread_cond_t both_finished_cv;
```

```
void WaitForBothFinished() {  
    pthread_mutex_lock(&lock);  
    while ( ) {  
        pthread_cond_wait(&both_finished_cv, &lock);  
    }  
    pthread_mutex_unlock(&lock);  
}
```

```
void Finish(int index) {  
    pthread_mutex_lock(&lock);  
    finished[index] = true;  
    -----  
    pthread_mutex_unlock(&lock);  
}
```

- A. `finished[0] && finished[1]`
- B. `finished[0] || finished[1]`
- C. `!finished[0] || !finished[1]`
- D. `finished[0] != finished[1]`
- E. something else

wait for both finished

// MISSING: init calls, etc.

```
pthread_mutex_t lock;  
bool finished[2];  
pthread_cond_t both_finished;
```

```
void WaitForBothFinished
```

```
pthread_mutex_lock(&lock)
```

```
while (_____
```

```
pthread_cond_wait(&both_finished_cv, &lock);
```

```
}  
pthread_mutex_unlock(&lock);
```

```
}
```

```
void Finish(int index) {
```

```
pthread_mutex_lock(&lock);
```

```
finished[index] = true;
```

```
_____  
pthread_mutex_unlock(&lock);
```

```
}
```

- A. pthread_cond_signal(&both_finished_cv)
- B. pthread_cond_broadcast(&both_finished_cv)
- C. if (finished[1-index])
 pthread_cond_signal(&both_finished_cv);
- D. if (finished[1-index])
 pthread_cond_broadcast(&both_finished_cv);
- E. something else

monitor exercise: one-use barrier

suppose we want to implement a one-use barrier; fill in blanks:

```
struct BarrierInfo {
    pthread_mutex_t lock;
    int total_threads; // initially total # of threads
    int number_reached; // initially 0
    -----
};
void BarrierWait(BarrierInfo *b) {
    pthread_mutex_lock(&b->lock);
    ++b->number_reached;
    if (b->number_reached == b->total_threads) {
        -----
    } else {
        -----
        -----
    }
    pthread_mutex_unlock(&b->lock);
}
```

backup slides

backup slides

deadlock versus starvation

starvation: one+ unlucky (no progress), one+ lucky (yes progress)

example: low priority threads versus high-priority threads

deadlock: no one involved in deadlock makes progress

deadlock versus starvation

starvation: one+ unlucky (no progress), one+ lucky (yes progress)

example: low priority threads versus high-priority threads

deadlock: no one involved in deadlock makes progress

starvation: once starvation happens, taking turns will resolve

low priority thread just needed a chance...

deadlock: once it happens, taking turns won't fix

abort and retry limits?

abort-and-retry

pthread's mutexes:

`pthread_mutex_trylock`

`pthread_mutex_timedlock`

how many times will you retry?

moving two files: abort-and-retry

```
struct Dir { mutex_t lock; HashMap entries; };  
void MoveFile(Dir *from_dir, Dir *to_dir, string filename) {  
    while (true) {  
        mutex_lock(&from_dir->lock);  
        if (mutex_trylock(&to_dir->lock) == LOCKED) break;  
        mutex_unlock(&from_dir->lock);  
    }
```

```
    Map_put(to_dir->entries, filename, Map_get(from_dir->entries, filename));  
    from_dir->entries.erase(filename);
```

```
    mutex_unlock(&to_dir->lock);  
    mutex_unlock(&from_dir->lock);  
}
```

Thread 1: MoveFile(A, B, "foo"); Thread 2: MoveFile(B, A, "bar")

moving two files: lots of bad luck?

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock) → LOCKED

trylock(&B->lock) → FAILED

unlock(&A->lock)

lock(&A->lock) → LOCKED

trylock(&B->lock) → FAILED

unlock(&A->lock)

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock) → LOCKED

trylock(&A->lock) → FAILED

unlock(&B->lock)

lock(&B->lock) → LOCKED

trylock(&A->lock) → FAILED

unlock(&B->lock)

livelock

livelock: keep aborting and retrying without end

like deadlock — no one's making progress
potentially forever

unlike deadlock — threads are not waiting

preventing livelock

make schedule random — e.g. random waiting after abort

make threads run one-at-a-time if lots of aborting

other ideas?

stealing locks???

how do we make stealing locks possible

unclean: just kill the thread

problem: inconsistent state?

clean: have code to undo partial operation

some databases do this

won't go into detail in this class

revokable locks?

```
try {  
    AcquireLock();  
    use shared data  
} catch (LockRevokedException le) {  
    undo operation hopefully?  
} finally {  
    ReleaseLock();  
}
```

deadlock detection

why? debugging or fix deadlock by aborting operations

idea: search for cyclic dependencies

detecting deadlocks on locks

let's say I want to detect deadlocks that only involve mutexes

goal: help programmers debug deadlocks

...by modifying my threading library:

```
struct Thread {  
    ... /* stuff for implementing thread */  
    /* what extra fields go here? */  
};  
  
struct Mutex {  
    ... /* stuff for implementing mutex */  
    /* what extra fields go here? */  
};
```

deadlock detection

why? debugging or fix deadlock by aborting operations

idea: search for cyclic dependencies

need:

- list of all contended resources

- what thread is waiting for what?

- what thread 'owns' what?

aside: divisible resources

deadlock is possible with divisible resources like memory,...

example: suppose 6MB of RAM for threads total:

- thread 1 has 2MB allocated, waiting for 2MB

- thread 2 has 2MB allocated, waiting for 2MB

- thread 3 has 1MB allocated, waiting for keypress

cycle: thread 1 waiting on memory owned by thread 2?

not a deadlock — thread 3 can still finish

- and after it does, thread 1 or 2 can finish

aside: divisible resources

deadlock is possible with divisible resources like memory,...

example: suppose 6MB of RAM for threads total:

- thread 1 has 2MB allocated, waiting for 2MB

- thread 2 has 2MB allocated, waiting for 2MB

- thread 3 has 1MB allocated, waiting for keypress

cycle: thread 1 waiting on memory owned by thread 2?

not a deadlock — thread 3 can still finish

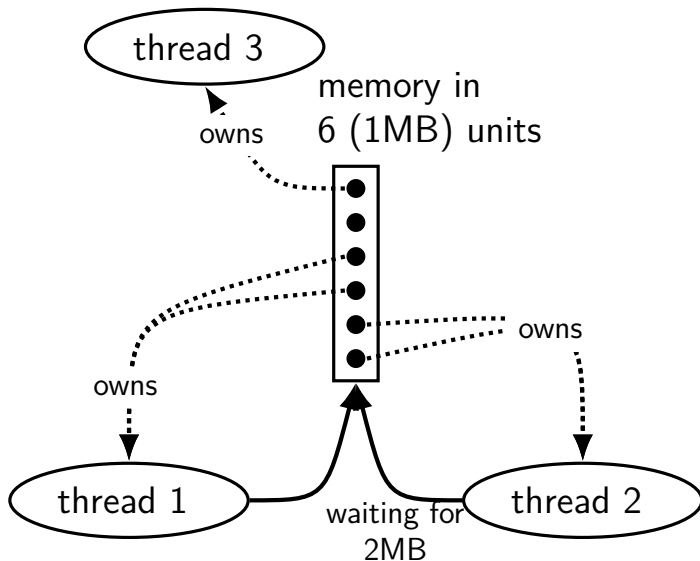
- and after it does, thread 1 or 2 can finish

...but would be deadlock

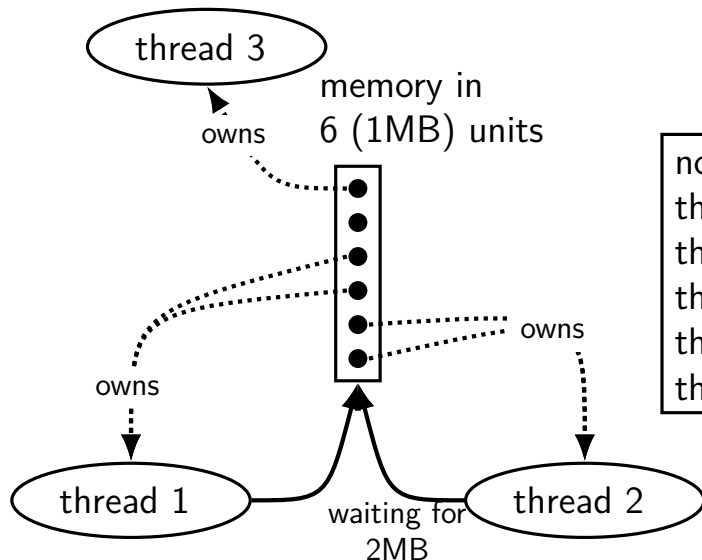
- ...if thread 3 waiting lock held by thread 1

- ...with 5MB of RAM

divisible resources: not deadlock

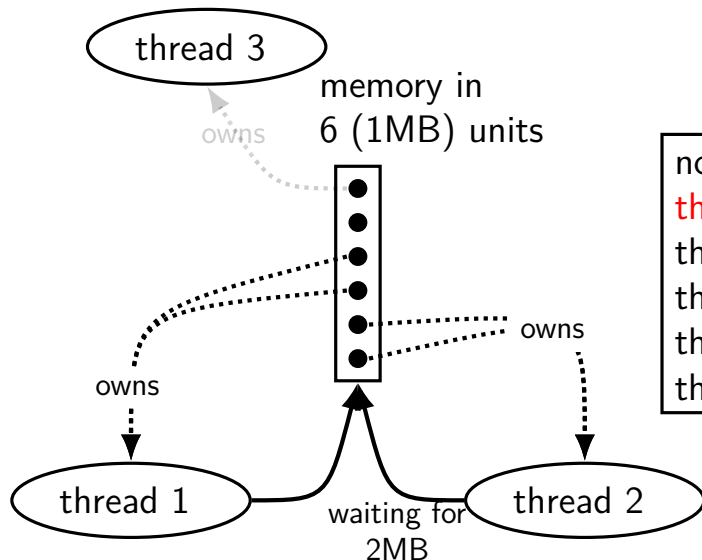


divisible resources: not deadlock



not deadlock:
thread 3 finishes
then thread 1 can get memory
then thread 1 finishes
then thread 2 can get resources
then thread 2 can finish

divisible resources: not deadlock



not deadlock:

thread 3 finishes

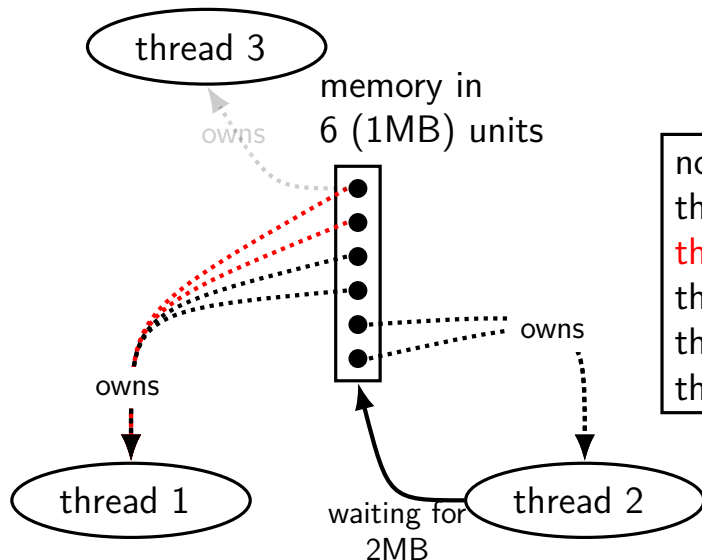
then thread 1 can get memory

then thread 1 finishes

then thread 2 can get resources

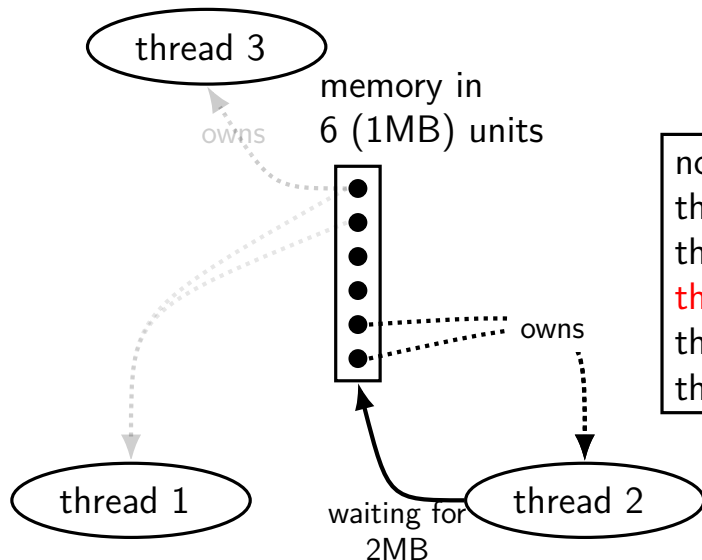
then thread 2 can finish

divisible resources: not deadlock



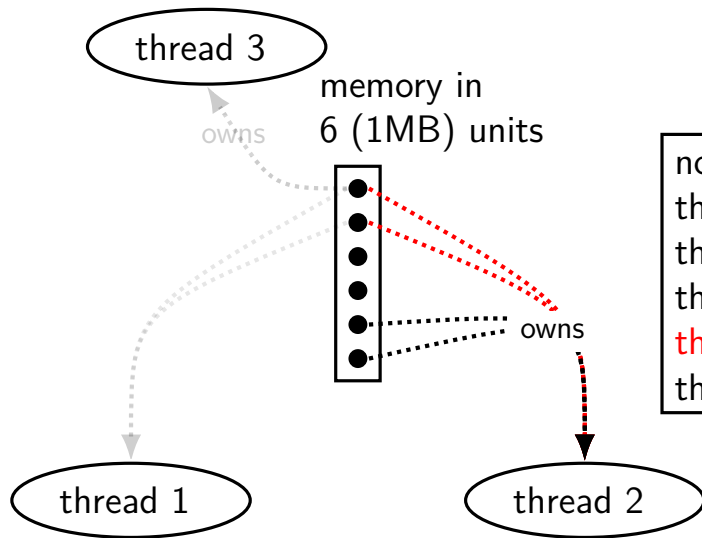
not deadlock:
thread 3 finishes
then thread 1 can get memory
then thread 1 finishes
then thread 2 can get resources
then thread 2 can finish

divisible resources: not deadlock



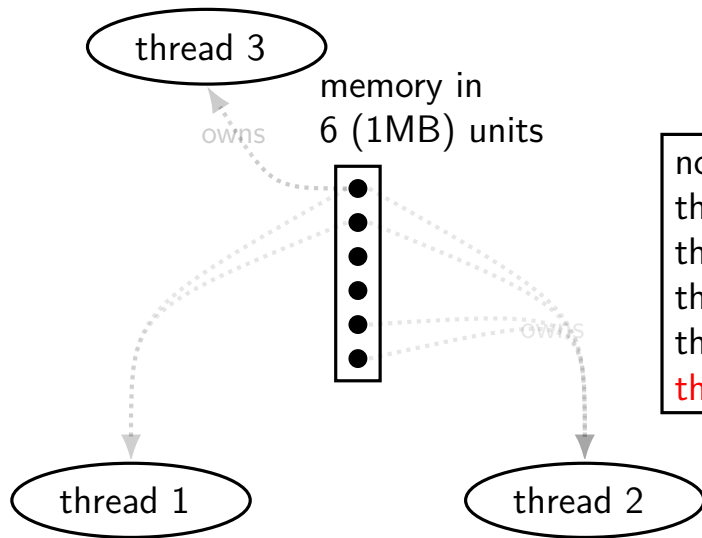
not deadlock:
thread 3 finishes
then thread 1 can get memory
then thread 1 finishes
then thread 2 can get resources
then thread 2 can finish

divisible resources: not deadlock



not deadlock:
thread 3 finishes
then thread 1 can get memory
then thread 1 finishes
then thread 2 can get resources
then thread 2 can finish

divisible resources: not deadlock



not deadlock:

thread 3 finishes

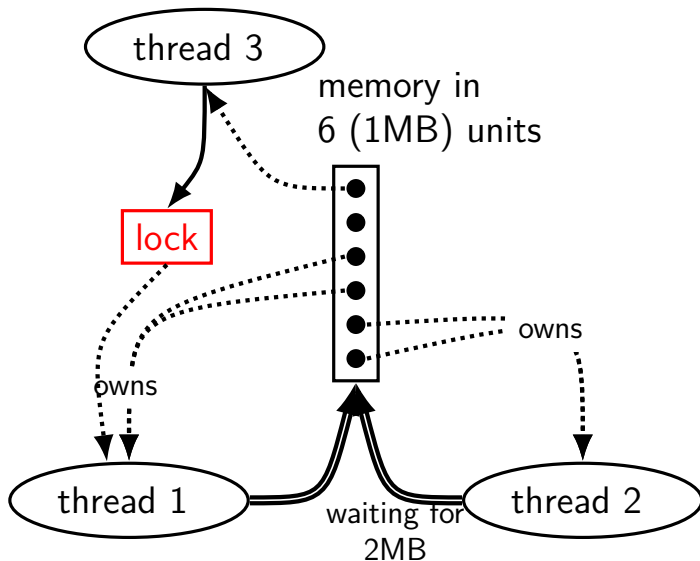
then thread 1 can get memory

then thread 1 finishes

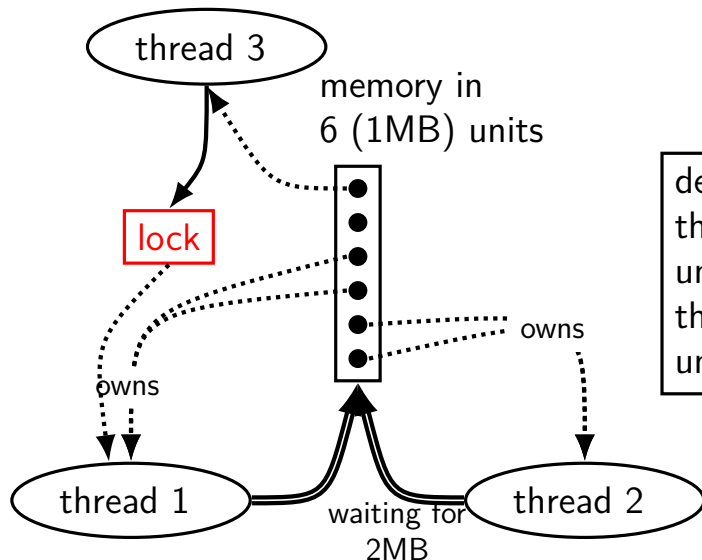
then thread 2 can get resources

then thread 2 can finish

divisible resources: is deadlock



divisible resources: is deadlock



deadlock:

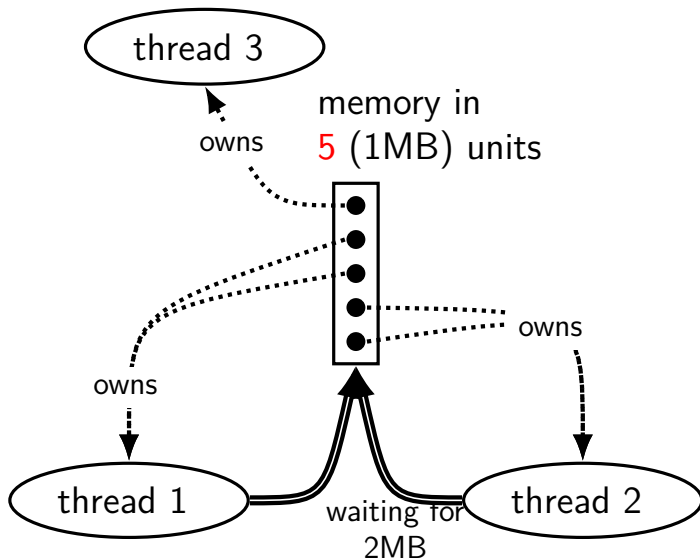
thread 3 can't finish

until thread 1 releases lock, but

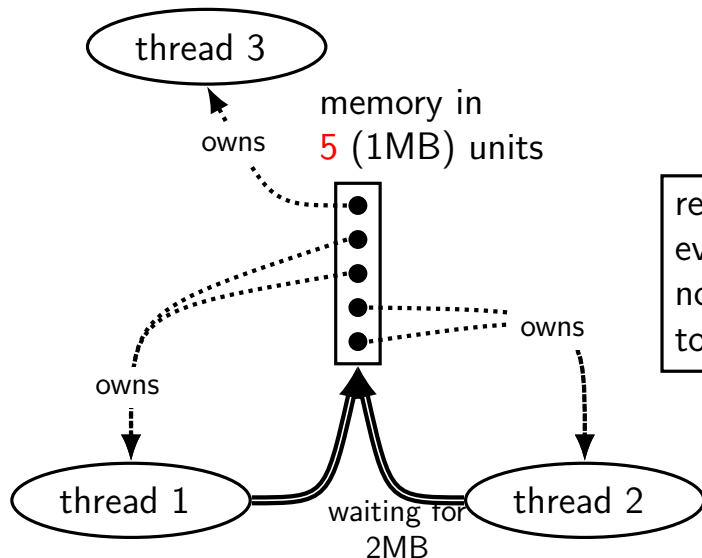
thread 1 can't finish

until thread 3 releases memory

divisible resources: is deadlock

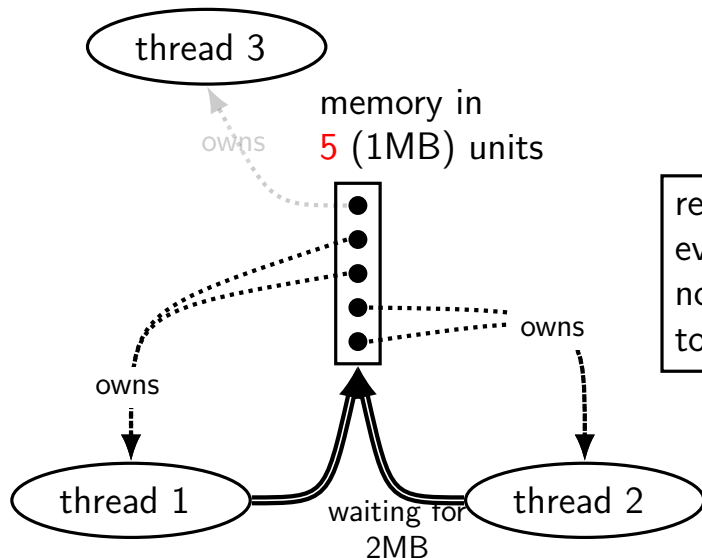


divisible resources: is deadlock



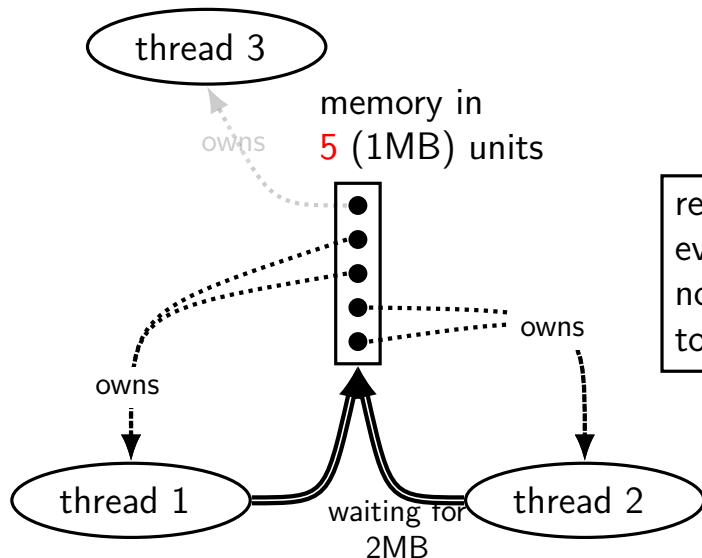
reducing memory: deadlock:
even after thread 3 finishes
no way for thread 1+2
to get what they want

divisible resources: is deadlock



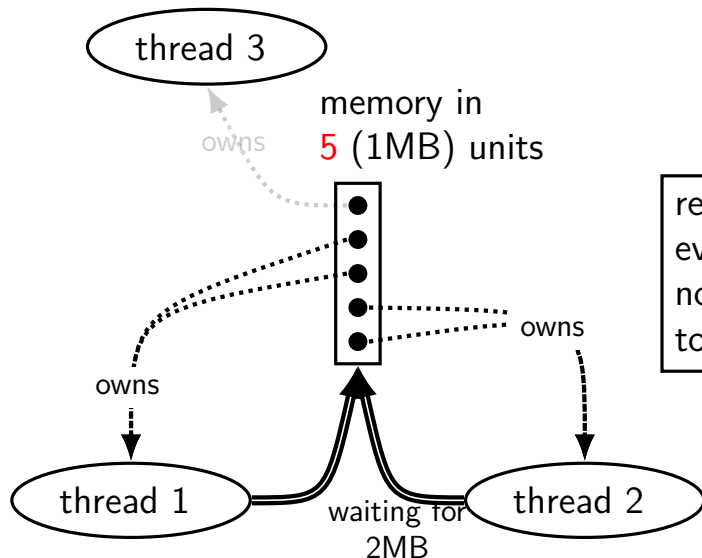
reducing memory: deadlock:
even after thread 3 finishes
no way for thread 1+2
to get what they want

divisible resources: is deadlock



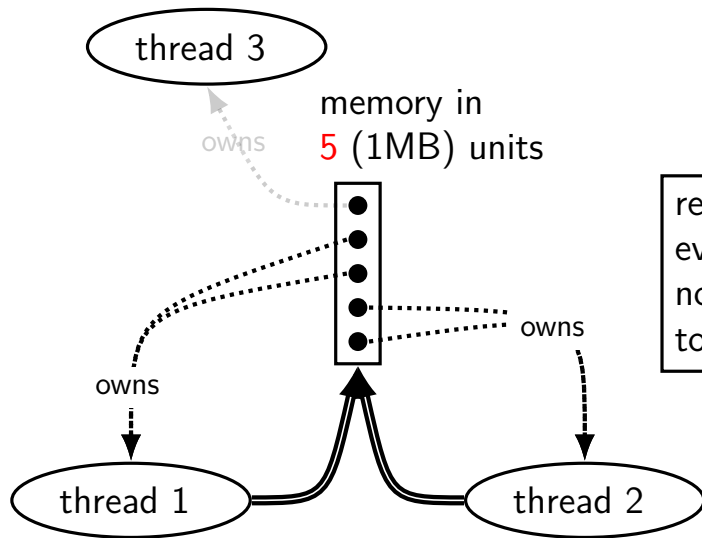
reducing memory: deadlock:
even after thread 3 finishes
no way for thread 1+2
to get what they want

divisible resources: is deadlock



reducing memory: deadlock:
even after thread 3 finishes
no way for thread 1+2
to get what they want

divisible resources: is deadlock



deadlock detection with divisible resources

for each resource: track which threads have those resources

for each thread: resources they are waiting for

repeatedly:

- find a thread where all the resources it needs are available

- remove that thread and mark the resources it has as free — it can complete now!

either: all threads eliminated *or* found deadlock

aside: deadlock detection in reality

requires:

- instrumenting contended resources

- “undo” to get out of deadlock

common example: for locks in a database

- database typically has customized locking code

- “undo” exists as side-effect of code for handling power/disk failures

related idea: *avoid* deadlock with detection on “what if” scenario

- see Banker’s algorithm

pipe() deadlock

BROKEN example:

```
int child_to_parent_pipe[2], parent_to_child_pipe[2];
pipe(child_to_parent_pipe); pipe(parent_to_child_pipe);
if (fork() == 0) {
    /* child */
    write(child_to_parent_pipe[1], buffer, HUGE_SIZE);
    read(parent_to_child_pipe[0], buffer, HUGE_SIZE);
    exit(0);
} else {
    /* parent */
    write(parent_to_child_pipe[1], buffer, HUGE_SIZE);
    read(child_to_parent_pipe[0], buffer, HUGE_SIZE);
}
```

This will **hang forever** (if HUGE_SIZE is big enough).

deadlock waiting

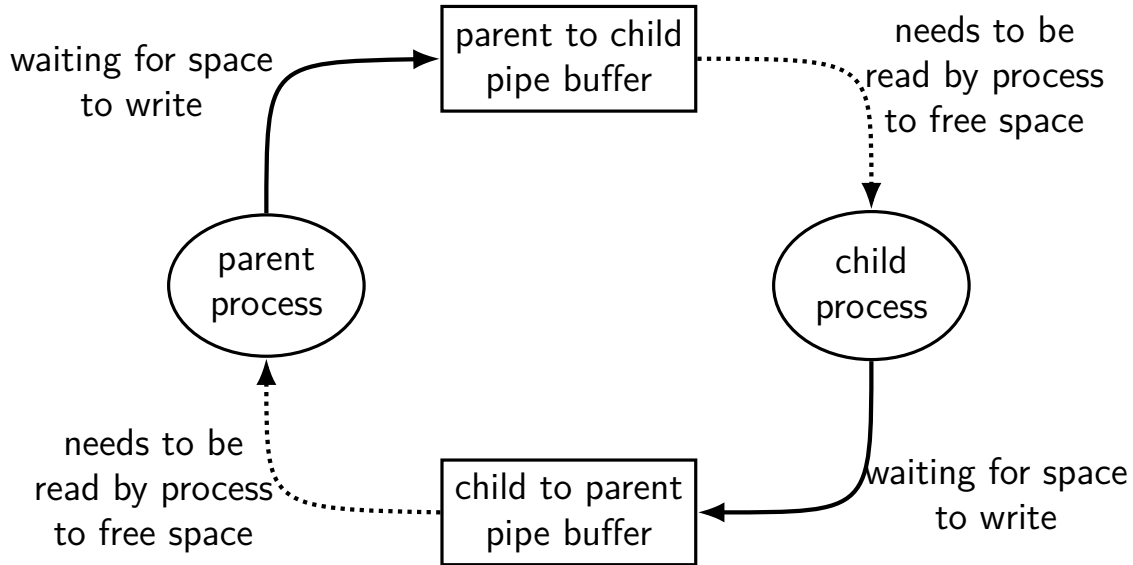
child writing to pipe waiting for free buffer space

...which will not be available until parent reads

parent writing to pipe waiting for free buffer space

...which will not be available until child reads

circular dependency



allocating all at once?

for resources like disk space, memory

figure out maximum allocation **when starting thread**

“only” need conservative estimate

only start thread if those resources are available

okay solution for embedded systems?

deadlock with free space

Thread 1

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB)
Free(1 MB)
```

Thread 2

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB)
Free(1 MB)
```

2 MB of space — deadlock possible with unlucky order

deadlock with free space (unlucky case)

Thread 1

AllocateOrWaitFor(1 MB)

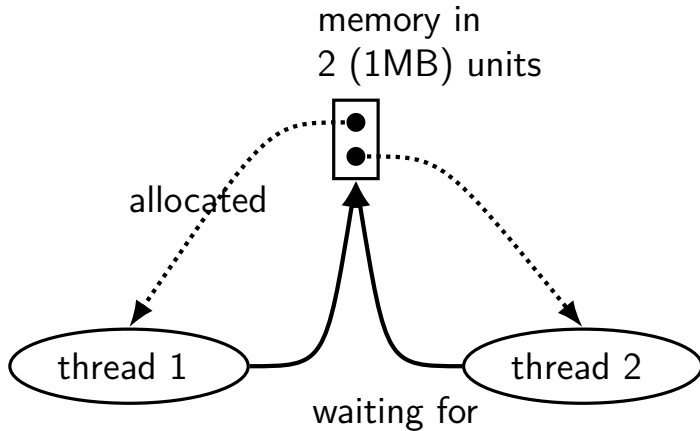
AllocateOrWaitFor(1 MB... stalled

Thread 2

AllocateOrWaitFor(1 MB)

AllocateOrWaitFor(1 MB... stalled

free space: dependency graph



deadlock with free space (lucky case)

Thread 1

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB);
Free(1 MB);
```

Thread 2

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB);
Free(1 MB);
```


AllocateOrFail

Thread 1

AllocateOrFail(1 MB)

AllocateOrFail(1 MB) **fails!**

Free(1 MB) (cleanup after failure)

Thread 2

AllocateOrFail(1 MB)

AllocateOrFail(1 MB) **fails!**

Free(1 MB) (cleanup after failure)

okay, now what?

- give up?

- both try again? — maybe this will keep happening? (called **livelock**)

- try one-at-a-time? — gaurenteed to work, but tricky to implement

AllocateOrSteal

Thread 1

AllocateOrSteal(1 MB)

AllocateOrSteal(1 MB)
(do work)

Thread 2

AllocateOrSteal(1 MB)

Thread killed to free 1MB

problem: can one actually implement this?

problem: can one kill thread and keep system in consistent state?

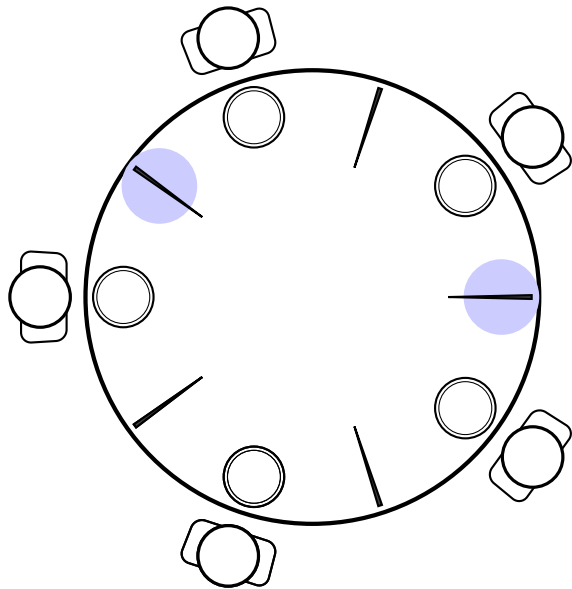
fail/steal with locks

pthread provides `pthread_mutex_trylock` — “lock or fail”

some databases implement *revocable locks*

- do equivalent of throwing exception in thread to ‘steal’ lock
- need to carefully arrange for operation to be cleaned up

dining philosophers — ordering

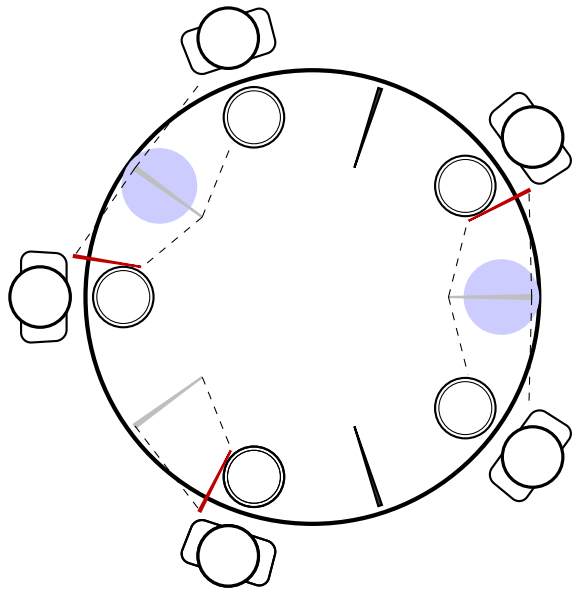


mark some chopsticks places

rule: grab from marked place first

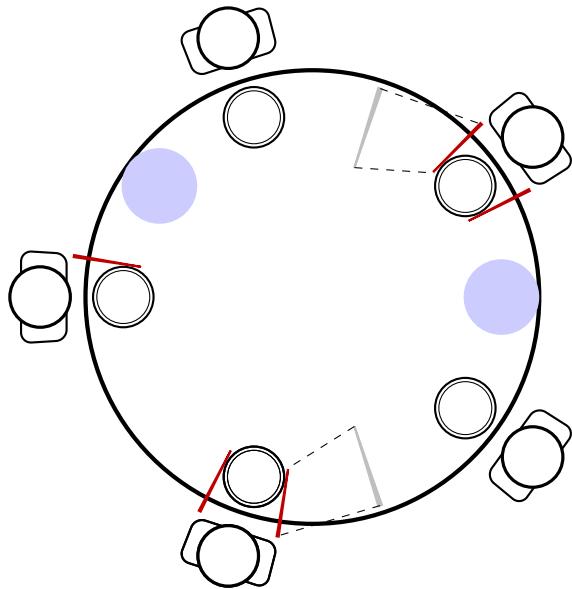
only grab other chopstick after that

dining philosophers — ordering



mark some chopsticks places
rule: grab from marked place first
only grab other chopstick after that

dining philosophers — ordering

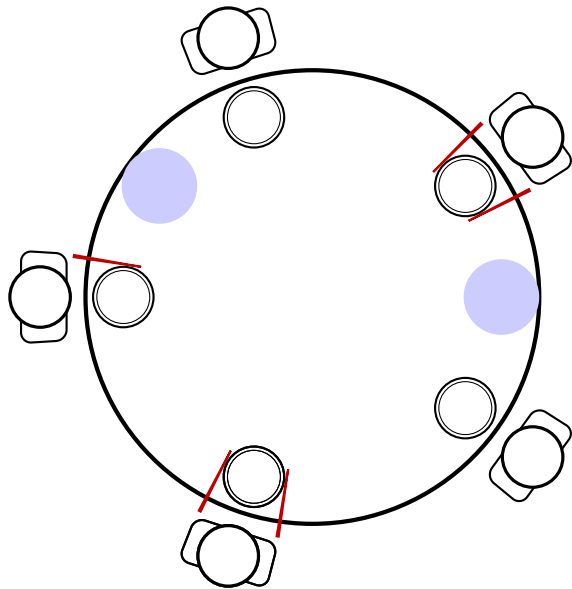


mark some chopsticks places

rule: grab from marked place first

only grab other chopstick after that

dining philosophers — ordering



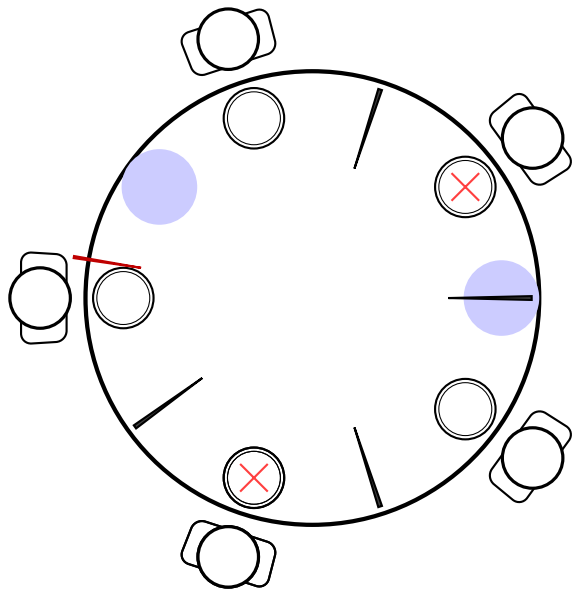
mark some chopsticks places

rule: grab from marked place first
only grab other chopstick after that

avoids circular dependency,
means everyone else

eventually gets a turn

dining philosophers — ordering

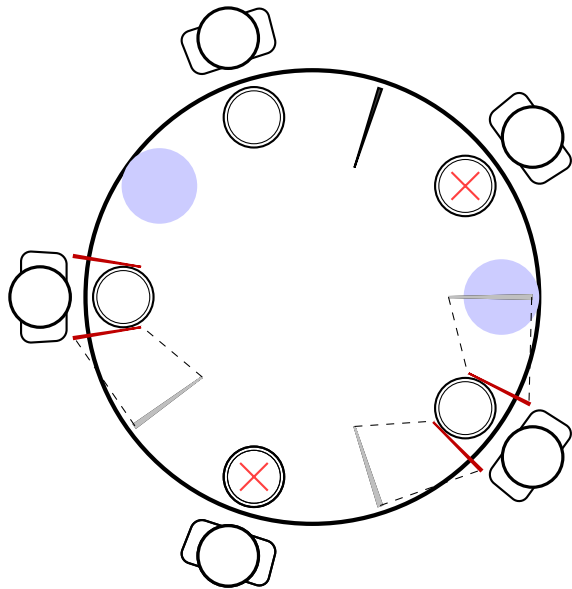


mark some chopsticks places

rule: grab from marked place first
only grab other chopstick after that

avoids circular dependency,
means everyone else
eventually gets a turn

dining philosophers — ordering



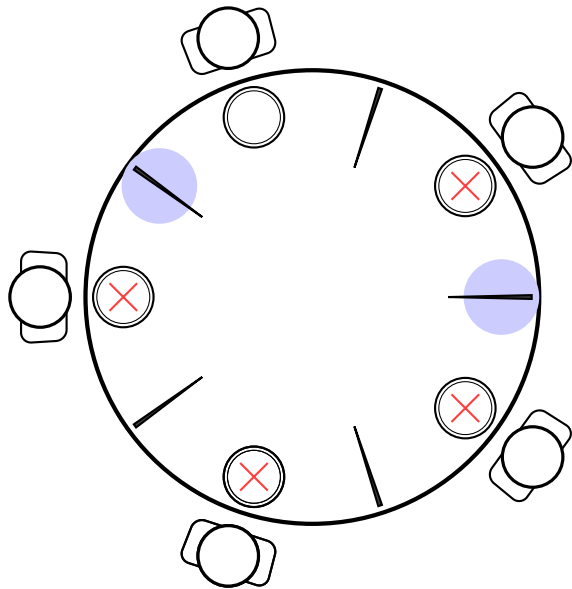
mark some chopsticks places

rule: grab from marked place first
only grab other chopstick after that

avoids circular dependency,
means everyone else

eventually gets a turn

dining philosophers — ordering

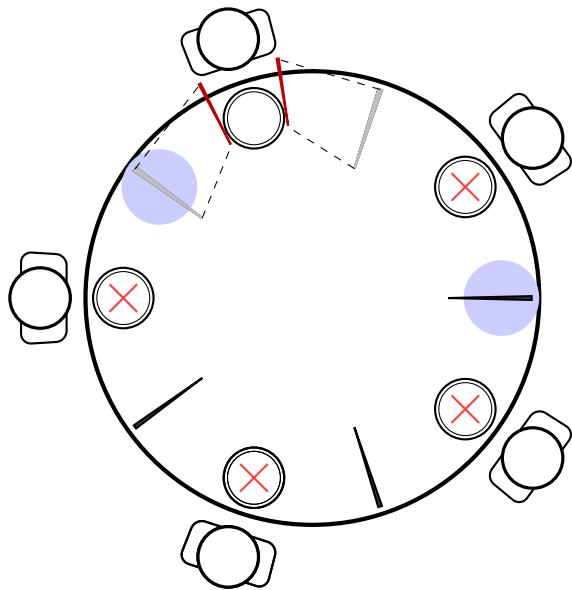


mark some chopsticks places

rule: grab from marked place first
only grab other chopstick after that

avoids circular dependency,
means everyone else
eventually gets a turn

dining philosophers — ordering

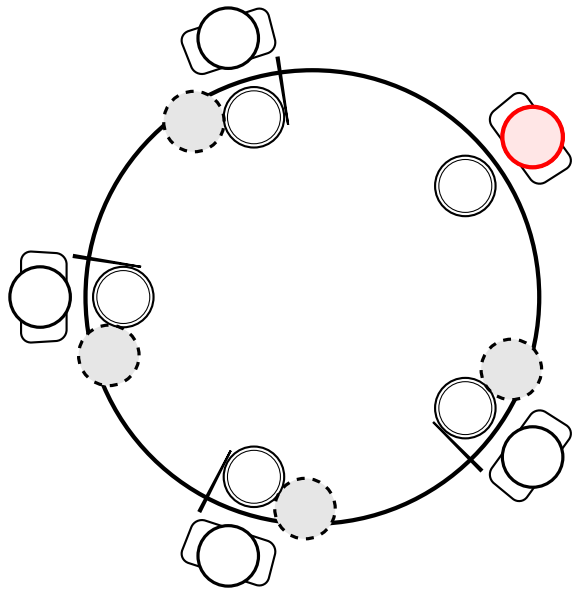


mark some chopsticks places

rule: grab from marked place first
only grab other chopstick after that

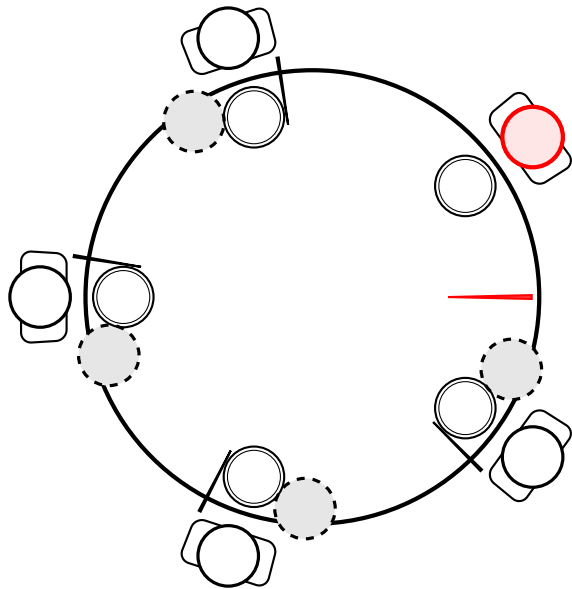
avoids circular dependency,
means everyone else
eventually gets a turn

dining philosophers — aborting



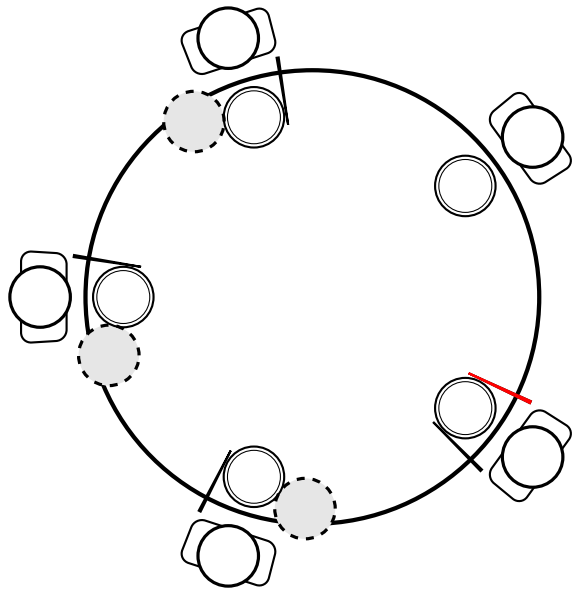
dining philosopher
what if someone's impatient
just gives up instead of waiting

dining philosophers — aborting



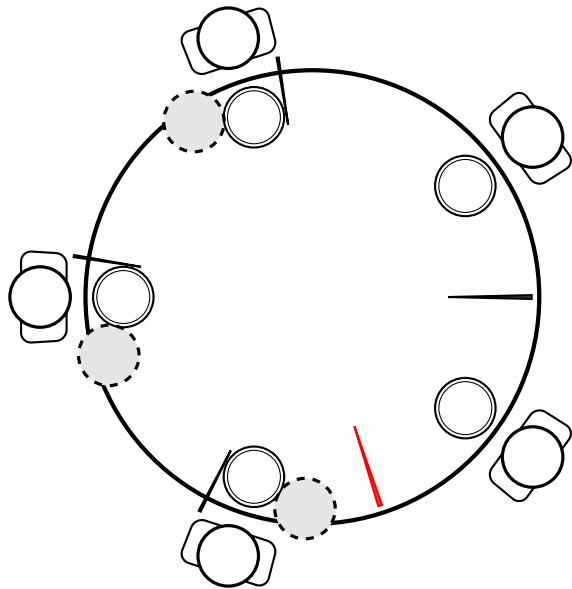
dining philosopher
what if someone's impatient
just gives up instead of waiting

dining philosophers — aborting



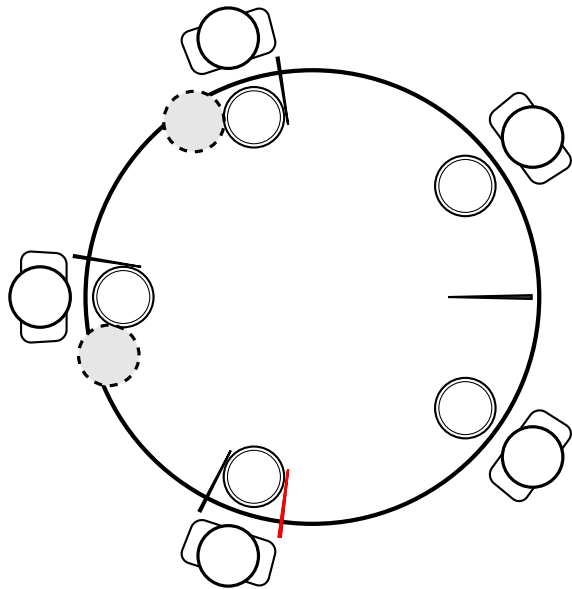
now everyone else can eat

dining philosophers — aborting



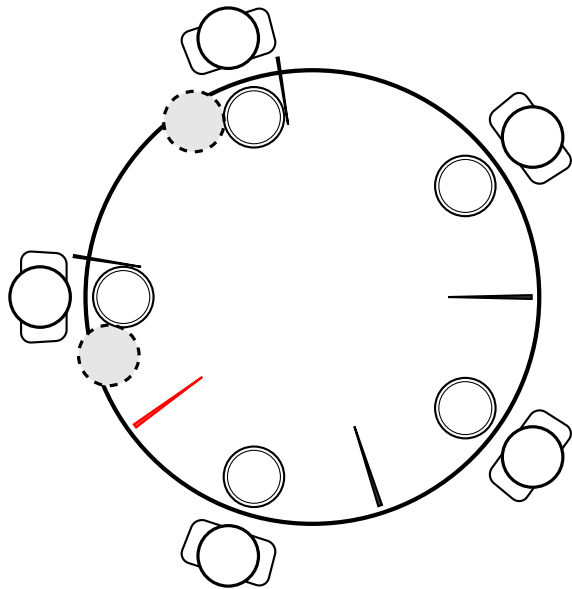
now everyone else can eat

dining philosophers — aborting



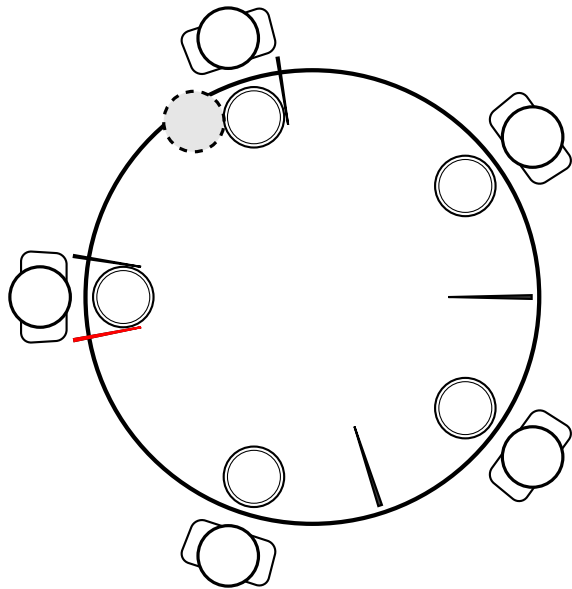
now everyone else can eat

dining philosophers — aborting



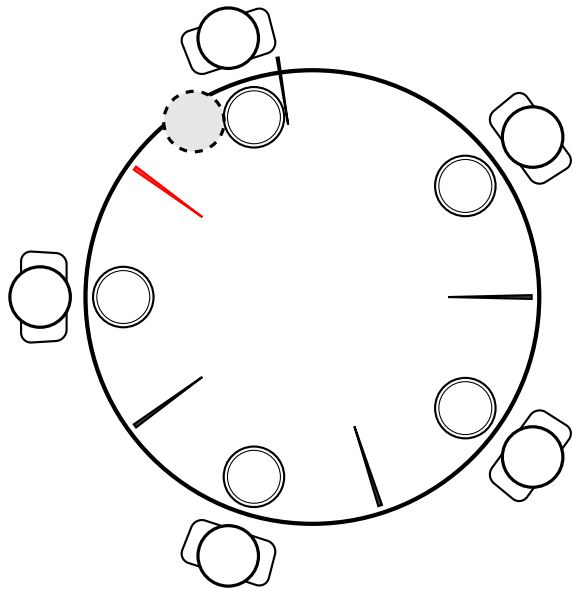
now everyone else can eat

dining philosophers — aborting



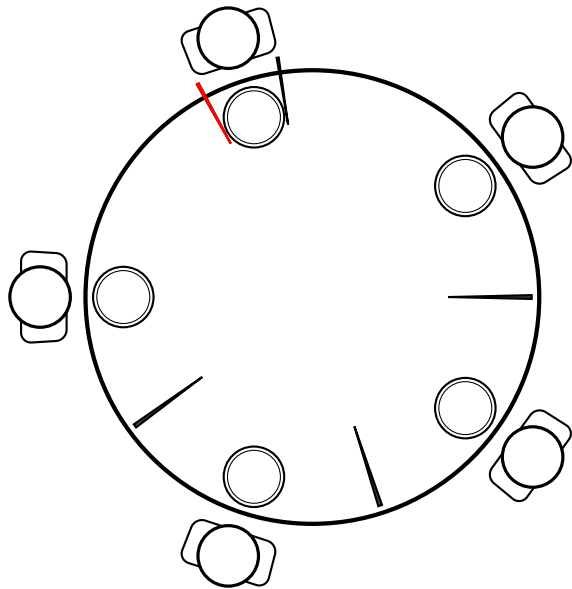
now everyone else can eat

dining philosophers — aborting



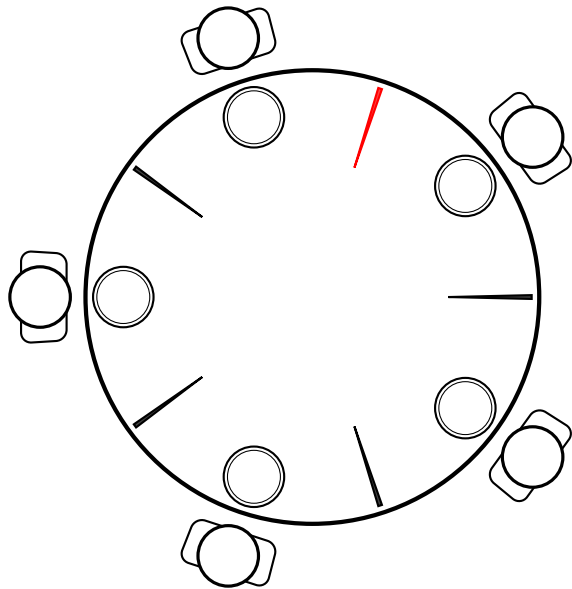
now everyone else can eat

dining philosophers — aborting



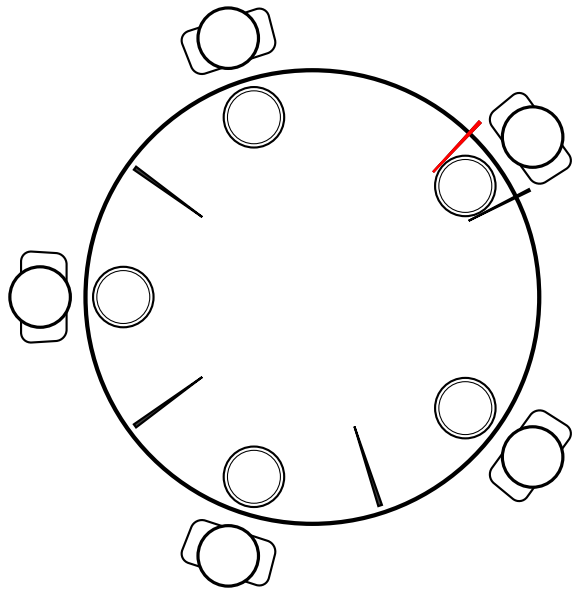
now everyone else can eat

dining philosophers — aborting



now everyone else can eat

dining philosophers — aborting



and person who gave up
might succeed later

using deadlock detection for prevention

suppose you know the *maximum resources* a process could request

make decision **when starting process** ("*admission control*")

using deadlock detection for prevention

suppose you know the *maximum resources* a process could request

make decision **when starting process** (*“admission control”*)

ask “what if every process was waiting for maximum resources”
including the one we’re starting

would it cause deadlock? then **don’t let it start**

called Banker’s algorithm